PROGRAMACIÓN · GRADO EN INGENIERÍA DE DATOS E IA

Programación en Python

De cero a estructuras de datos, en orden y con ejemplos

Introducción a la construcción de software · 01 Literales, tipos, variables · 02 Sentencias básicas · 03 Funciones · 04 Estructuras de datos · Cuadernos 02 y 03 · Hojas de ejercicios · docs.python.org · Data Structures

He juntado todos los PDFs y la página oficial de estructuras de datos de Python en un solo sitio. Lo que estaba repetido sale una única vez. Todos los ejercicios están resueltos y todo el código está probado en Python 3.13. Los errores de las transparencias están corregidos (los tienes listados al final).

Índice

1 **Programar y construir software**
Programa, algoritmo · Von Neumann · codificación binaria · lenguajes · compilar o interpretar · paradigmas · fases · calidad

2 **Primeros pasos con Python**
Qué es Python · cómo ejecutarlo · hola mundo · input() y print() · Python como calculadora

3 **Los ladrillos del lenguaje**
Análisis léxico · identificadores · literales · tipos · objetos y variables · operadores · booleanos · cadenas · expresiones y sentencias

4 **Sentencias de control**
Algoritmos y programación estructurada · if/elif/else · for y range · while · break y continue · ejercicios resueltos

5 **Funciones**
def y return · pila de llamadas · tipos de parámetros · ámbito · paso de argumentos · recursividad · lambda · main · ejercicios

6 **Estructuras de datos**
Secuencias y slicing · cadenas · tuplas · range · listas · pilas y colas · comprensiones · del · conjuntos · diccionarios · bucles · comparaciones

7 **Programas con estructuras de datos**
Suma de cuadrados recursiva · divisores propios · primos · números amigos · números romanos

8 **Chuleta**
Todo lo importante en dos páginas

9 **Erratas corregidas**
Lo que estaba mal en el material

CÓMO LEER ESTA GUÍA

Definición lo que tienes que saber decir. Analogía una imagen para entenderlo. Ejemplo resuelto. Ejercicio con su solución. ¡Ojo! trampas típicas. Truco atajos.

En el código, las líneas que empiezan por `>>>` son lo que escribes en el intérprete interactivo y las líneas en **verde** son lo que Python responde.

1 Programar y construir software

1.1 Ingeniería y programación

Los ingenieros diseñan y construyen máquinas: un coche, un circuito, un puente. Los **ingenieros de software** programan una máquina de **propósito general** (el ordenador) y con ella crean máquinas **inmateriales**: programas que hacen una tarea concreta.

Los límites de la programación son la **imaginación** y la **fiabilidad**. El ordenador **no entiende** las instrucciones: las **ejecuta**. Todas, y exactamente como están escritas.

ANALOGÍA · LA HABITACIÓN CHINA (JOHN SEARLE)

Imagina a alguien encerrado en una habitación que no sabe chino. Le pasan papeles con símbolos chinos y tiene un libro de reglas que dice "si recibes estos símbolos, devuelve estos otros". Desde fuera parece que entiende chino, pero solo sigue reglas sin comprender nada. Eso es un ordenador: sigue tus instrucciones al pie de la letra, aunque no tengan sentido.

1.2 Definiciones básicas

DEFINICIONES

Computadora: máquina de propósito general formada por muchos componentes: **hardware** (la parte física) y **software** (los programas).

Programación: actividad que permite crear programas de computadora.

Programa: lista de instrucciones bien definidas diseñada para ser ejecutada en una computadora.

Algoritmo: lista de instrucciones bien definidas para hacer una tarea en un **tiempo finito**.

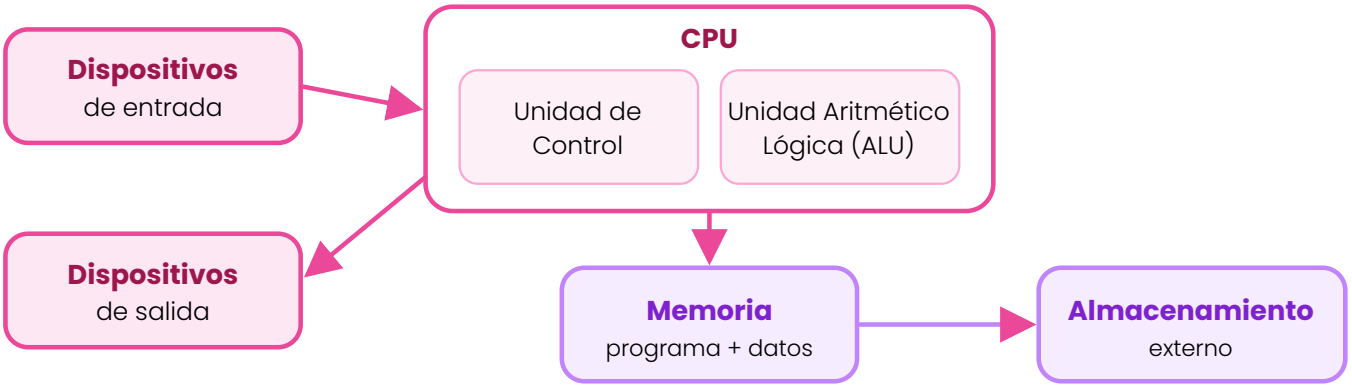


ANALOGÍA

Un **algoritmo** es una receta (la idea: "bate los huevos, añade harina…"). Un **programa** es esa receta escrita en un idioma concreto que la cocina entiende (Python, C…). Un mismo algoritmo puede escribirse como muchos programas distintos.

1.3 El modelo mental: la máquina de Von Neumann

Para programar conviene imaginar el ordenador así:



Parte	Para qué sirve
Entrada / salida	Comunicarse: teclado, pantalla, joystick, sensores, redes…
Unidad de control	Dirige: recoge la siguiente instrucción, la decodifica y manda ejecutarla
ALU	Calcula: comparar, sumar, restar, mover…
Memoria	Guarda **a la vez el programa y los datos**
Almacenamiento externo	Guardar y recuperar a largo plazo: discos duros, cintas, la nube…

♥ El ciclo de instrucción

1. Recoge la siguiente instrucción → **2. Decodifica** ¿qué es? → **3. Ejecuta** en la ALU → **4. Almacena** el resultado ↻

1.4 Memoria y codificación binaria

La capacidad de **guardar información** y la de **ejecutar instrucciones** son las dos características principales de un ordenador.

- La memoria está formada por **unidades de almacenamiento con dirección**. Nosotros les damos **nombres** mediante **variables**.
- Cada unidad guarda una secuencia de unos y ceros.
- Esa secuencia se puede **interpretar de formas muy distintas** según dónde esté y cómo se haya codificado.

Tipo de información	Cómo se codifica en binario
Números naturales	En base 2: $9_{10} = 1001_2$
Números enteros	Complemento a uno o complemento a dos
Números reales	Estándar **IEEE 754** en precisión simple o doble
Booleanos	0 = falso; distinto de 0 = verdadero
Caracteres	Antes **ASCII**. Hoy **Unicode**: cada carácter tiene un número (*code point*) y se guarda con una codificación como **UTF-8** o UTF-16 (otras: Windows-1252, Latin1…)
Imágenes, sonido…	Cada formato tiene su codificación: jpg, png, mp3…

Los lenguajes de alto nivel nos dejan trabajar **sin pensar en los bits**: escribimos números y textos y ellos se ocupan de codificarlos.

1.5 Lenguajes de programación

DEFINICIÓN

Lenguaje de programación: notación que permite escribir instrucciones bien definidas para que las ejecute una computadora.

Según su **nivel de abstracción** (lo lejos que están de los bits):

Nivel	Ejemplo	Cómo es
Lenguaje máquina	`1000100111000011`	Binario puro: lo que entiende la CPU
Ensamblador	`movl %eax %ebx`	Nemónicos que corresponden 1 a 1 con el código máquina
Alto nivel	`if (hoja == raiz): ...`	Cercano al lenguaje humano

Cada CPU tiene su propio lenguaje máquina y su ensamblador. ¿Habría que reescribir cada programa para cada máquina? No: los **lenguajes de alto nivel** junto con los **compiladores e intérpretes** resuelven ese problema.

♥ Compilados frente a interpretados

COMPILADOS

El programa entero se **traduce de una vez** a otro lenguaje (normalmente ensamblador o código objeto) y luego se ejecuta la traducción. Suelen ser **más eficientes**. Ej.: C.

INTERPRETADOS

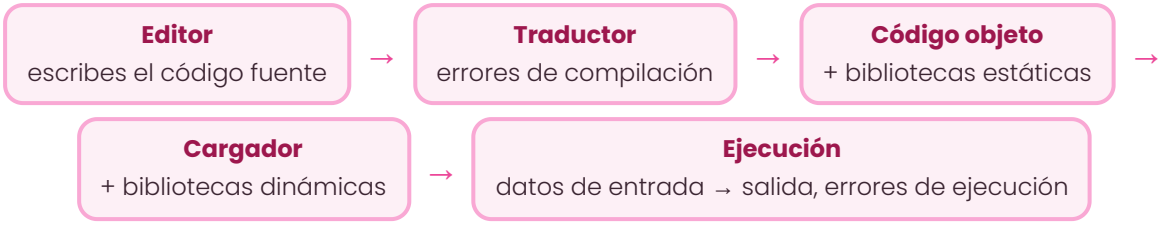
El código fuente se ejecuta **instrucción a instrucción** en una **máquina virtual**, que es la que se adapta a cada máquina real. Permiten **probar mejor** y son más **flexibles** para cambiar de arquitectura. Ej.: **Python**.

ANALOGÍA

Compilar es **traducir un libro entero** antes de publicarlo: cuesta al principio, pero luego se lee rapidísimo. Interpretar es tener un **intérprete simultáneo** en una conferencia: traduce frase a frase en directo; es más lento, pero puedes cambiar el discurso sobre la marcha.

Otras ideas de las diapositivas: cualquier algoritmo podría implementarse directamente en hardware, y los lenguajes máquina son de bajo nivel para ahorrar costes y lograr flexibilidad y velocidad.

♥ Del código fuente a la ejecución



Hay dos momentos: el **tiempo de compilación** (estático: antes de ejecutar) y el **tiempo de ejecución** (dinámico: mientras el programa funciona). Los errores pueden aparecer en cualquiera

de los dos.

♥ Tipos de traductores

Traductor	Lenguaje fuente → Lenguaje objeto
Preprocesador	Lenguaje de alto nivel "ampliado" → el mismo lenguaje estándar
Compilador	Lenguaje de alto nivel → lenguaje máquina o ensamblador
Ensamblador	Ensamblador → lenguaje máquina
Enlazador	Mismo lenguaje en los dos lados: une y reubica trozos de código para obtener un ejecutable

♥ Fases de la traducción de un lenguaje de alto nivel

Análisis léxico (scanner) → tokens → **Análisis sintáctico** (parser) → árbol sintáctico → **Análisis semántico** → árbol semántico →

Código intermedio → **Optimizador** → **Generador de código** → código objeto

Durante todo el proceso se usan una **tabla de símbolos** (nombres de variables, funciones…), una **tabla de literales** y un **gestor de errores**.

ANALOGÍA

Es como corregir una redacción: primero separas las **palabras** (léxico), luego compruebas que las **frases estén bien construidas** (sintáctico) y por último que **tengan sentido** (semántico): "El gato come rápido" está bien; "Rápido el come gato" falla en la sintaxis; "La piedra come felicidad" tiene buena sintaxis pero falla en la semántica.

1.6 Paradigmas de programación

Un paradigma es una **forma de pensar** los programas. Se aplica a los lenguajes de alto nivel.

Paradigma	Idea principal	Lenguajes (año)
Imperativo (o procedimental)	Se calcula ejecutando **órdenes que cambian el estado** de la máquina (memoria, entrada/salida, pila de llamadas y sentencia que toca). Se centra en el **proceso**: qué hay que calcular y cómo.	FORTRAN (1954), COBOL (1959), BASIC (1963), PL/I (1971), Pascal (1974), C (1979), ADA (1979), **Python (1989)**
Estructurado	Usa solo tres construcciones: **secuencia, selección y repetición**, más subrutinas/funciones.	
Orientado a objetos	Lo importante es **la información**, no el proceso. **Objeto = datos + operaciones**. Los objetos se pasan **mensajes**. Muy relacionado con la **programación orientada a eventos** (un bucle infinito que recoge eventos de una cola y los gestiona).	SIMULA67 (1968), Smalltalk (1971), C++ (1985), Object Pascal (1986), Eiffel (1988), **Python (1989)**, Java (1995)
Declarativo	Se dice **qué resultado se quiere**, no cómo conseguirlo. Subtipos: lógica (Prolog), restricciones, SQL, XSLT, informes RPG.	RPG (1959), expresiones regulares/grep (1960s), Prolog (1972), SQL (1986), Oz (1991), XSLT (1999)
Funcional	Se calcula **combinando funciones** sin efectos colaterales. Puede verse como un tipo de declarativo.	LISP (1956), ADL (1959), Scheme (1975), Common LISP (1984), **Python (1989)**, Haskell (1990)
Concurrente	Varios procesos a la vez: **paralelos** (una máquina, varios procesadores) o **distribuidos** (varias máquinas). Es transversal a los demás.	Pascal Concurrente (1975), ADA (1979), Occam (1983), Erlang (1986), Java (1995)

ANALOGÍA · PEDIR UNA PIZZA

Imperativo: "amasa, extiende, pon tomate, hornea 12 min". **Declarativo**: "quiero una margarita" (y el pizzero sabe cómo). **Orientado a objetos**: le mandas un mensaje al objeto *pizzería*: `pizzeria.pedir("margarita")`.

TRUCO

Fíjate en que **Python aparece en tres tablas**: es **multiparadigma** (imperativo, orientado a objetos y con aspectos funcionales).

♥ Dominios de aplicación

Computación científica (la primera históricamente) Sistemas de gestión (el dominio más grande) Software de sistemas (S.O., compiladores, máquinas virtuales, cloud) Entretenimiento Simulación Web Computación móvil Tiempo real Software empotrado Análisis masivo de datos (Data Mining) Inteligencia Artificial (demostradores, agentes, robótica, procesamiento del lenguaje)

1.7 Construir software

Se habla de programación "a lo pequeño" frente a "a lo mediano" o "a lo grande". La construcción de programas grandes es la **Ingeniería del Software**, pero la **programación** es la única tarea imprescindible.

♥ Procesos para crear una aplicación

#	Proceso	Pregunta que responde
1	**Análisis**	¿Qué hay que hacer? (qué resolver, límites, restricciones)
2	**Especificación**	Fijar exactamente lo que hará el programa
3	**Diseño**	¿Cómo hacerlo? Organizar la solución, diseñar los algoritmos, dividir en partes, refinamiento progresivo, modularidad, reutilización; criterios de **cohesión y acoplamiento**
4	**Implementación** (codificación)	Hacerlo: escribir el código
5	**Verificación y prueba**	Probarlo con distintos tests para confiar en que está bien; si falla, **depurar** y corregir
6	**Mantenimiento**	Cambiarlo: corregirlo, perfeccionarlo, adaptarlo

La **fase de construcción** comprende fundamentalmente: **diseño, implementación y prueba**.

COHESIÓN Y ACOPLAMIENTO

Buen diseño = **alta cohesión** (cada parte hace una sola cosa y la hace bien) y **bajo acoplamiento** (las partes dependen poco unas de otras). Como un equipo donde cada persona tiene una tarea clara y no necesita preguntar a los demás a cada rato.

♥ Criterios de calidad del software

Básicos	
Corrección (eficacia)	¿Hace lo que quiero? ¿Resuelve el problema?
Robustez	¿Reacciona bien ante situaciones excepcionales?
Fiabilidad	¿Lo hace sin errores?
Eficiencia	¿Usa bien la memoria y el tiempo?

Otros	
Extensibilidad	¿Se puede ampliar fácilmente?
Reutilización	¿Puedo aprovechar partes?
Verificabilidad	¿Es fácil de probar?
Compatibilidad	¿Se combina con otro software?
Facilidad de uso	¿El usuario lo maneja bien?
Reparabilidad	¿Es fácil arreglarlo?
Portabilidad	¿Funciona en otra máquina?
Funcionalidad	¿Ofrece lo adecuado?
Oportunidad	¿Sale a tiempo al mercado?
Integridad	¿Solo accede quien debe?
Economía	¿Se ha cumplido el presupuesto?

TRUCO PARA NO CONFUNDIRLAS

Robustez = aguanta lo inesperado (si escribes letras donde pide un número, no explota). **Fiabilidad** = con datos normales, no se equivoca. **Corrección** = hace lo que tenía que hacer.

2 Primeros pasos con Python

2.1 ¿Qué es Python?

- Un lenguaje de **alto nivel** e **interpretado**.
- **Multiparadigma**: tiene programación imperativa, orientada a objetos y funcional.
- Con **tipado dinámico**: no declaras el tipo de las variables; se sabe por el valor que guardan.
- **Multiplataforma**: funciona en Windows, Linux, Mac...
- Pertenece a la **Python Software Foundation (PSF)**, con licencia libre (PSF License): permite usarlo, copiarlo, distribuirlo y hacer obras derivadas.

♥ Breve historia

1. Empezó a desarrollarlo **Guido van Rossum en 1989**.
2. El nombre viene de los **Monty Python** (¡no de la serpiente!).
3. Primera versión estable (1.0) en 1994. Hoy van por la 3.13 y siguientes.
4. Desde la versión 2.1, la propiedad intelectual es de la PSF.
5. La distribución oficial es **CPython**, pero hay muchas más.

¡OJO CON LAS VERSIONES!

Conviven dos familias **incompatibles**: **Python 2.7** (abandonada, sin mantenimiento) y **Python 3**. Usa siempre Python 3. Si encuentras en internet `print "hola"` sin paréntesis, es Python 2.

2.2 Cómo ejecutar Python

Hay dos formas de trabajar: escribir un fichero `.py` con un editor y dárselo al intérprete, o hablar con el intérprete en una **sesión interactiva**. En los dos casos el intérprete lee de la entrada estándar (**stdin**, el teclado) y escribe en la salida estándar (**stdout**, la pantalla).

Entorno	Cómo
Terminal	`python fichero.py` · `python -c "comandos"` · `python` (abre el intérprete interactivo)
Otros intérpretes	IPython
Entornos gráficos	Visual Studio Code, Spyder, Jupyter...
En la web	Colab, mycompiler, PyScript...

2.3 Tu primer programa

```
# El hola mundo en Python
print("Hola mundo.")
```

```
# Hola mundo con entrada desde stdin
nombre = input()
print("Hola " + nombre + ".")
```

Si en el segundo escribes *Lucía*, imprime `Hola Lucía.`. El `#` empieza un **comentario**: Python lo ignora.

♥ input(): leer del teclado

`input()` lee una línea completa de la entrada estándar y la devuelve **siempre como cadena (str)**. Si quieres un número, hay que convertirlo:

```
entrada = input()          # el usuario escribe 5
type(entrada)              <class 'str'>    ← ¡es el texto "5"!
numero = int(entrada)      # ahora sí es el entero 5
type(numero)               <class 'int'>
altura = float(input("Altura: "))   # con texto de aviso y a float directamente
```

♥ print(): escribir en pantalla

`print()` envía su contenido a la salida estándar. Admite **varios argumentos** y los separa con un espacio; al terminar, salta de línea. Para pegar texto y números con `+`, el número debe pasarse a cadena con `str()`:

```
print("El número leído ha sido: " + str(numero))   El número leído ha sido: 5
print("El número leído ha sido:", numero)          El número leído ha sido: 5
print("a", "b", "c", sep="-", end="!\n")            a-b-c!
```

`sep` cambia el separador y `end` lo que se escribe al final (por defecto `"\n"`). Es lo que en el cuaderno se dice que "veremos más adelante".

2.4 Python como calculadora

```
>>> 2 + 3 - 4 + 4 * 5
21
>>> ((2 + 3) - 4) + (4*5)          # así lo agrupa Python: igual
21
>>> 2 + (3 - (4 + (4*5)))          # otra agrupación: distinto
-19
```

Python aplica las reglas de **precedencia** (primero `*`, luego `+` y `-`) y la **asociatividad de izquierda a derecha**. Excepción: la potencia `**` se evalúa **de derecha a izquierda**:

```
>>> 8 ** 5 ** 4 == 8 ** (5 ** 4)    # es 8 elevado a 625: ¡un número de 565 cifras!
True
>>> (8 ** 5) ** 4
1152921504606846976
```

Los enteros se pueden escribir en varias bases, y las comparaciones devuelven booleanos:

```
>>> 16 == 0b10000 and 16 == 0o20 and 16 == 0x10
True
>>> 10/3              # / siempre da un real
3.3333333333333335
>>> int(_)            # _ = el último resultado (solo en modo interactivo)
3
>>> type(_)           # type() dice el tipo de algo
<class 'int'>
>>> True and (False or not False)
True
```

¡OJO!

Fíjate en el último dígito de `3.3333333333333335`: los reales se guardan en binario con precisión limitada (IEEE 754), así que a veces no son exactos. Otro ejemplo famoso: `0.1 + 0.2` da `0.30000000000000004`. Por eso los reales no se comparan con `==` (lo verás en la función `son_iguales` del capítulo 5).

3 Los ladrillos del lenguaje

Objetivos del tema: entender cómo Python "lee" un programa (análisis léxico) y aprender qué son los **literales, tipos, variables, operadores, expresiones y sentencias**.

3.1 Cómo lee Python tu programa: el análisis léxico

Lo primero que hace el intérprete es un **análisis léxico**: convierte el texto en una secuencia de **tokens** (las "palabras" del lenguaje). Python lee texto en **Unicode**, por defecto con codificación **UTF-8** (se puede declarar otra).

♥ Líneas lógicas y físicas

- **Línea física**: lo que ves en el editor, una secuencia de caracteres que acaba en un fin de línea. Se aceptan los tres finales: el de Linux (LF), el de Windows (CR LF) y el del Mac antiguo (CR). El final del fichero también termina una línea física.
- **Línea lógica**: lo que Python considera una sentencia; termina con el token **NEWLINE**. Cada sentencia simple debe ir en una línea lógica (las compuestas ocupan varias).
- Para **unir varias líneas físicas en una lógica**, pon una barra invertida `\` al final:

```
total = 1 + 2 + 3 + \
        4 + 5           # es una sola línea lógica: total = 15
```

Dentro de paréntesis, corchetes o llaves también se puede partir la línea sin `\` (por ejemplo entre los argumentos de un `print`).

♥ Comentarios

Empiezan con `#` y llegan hasta el final de la línea física. El intérprete los ignora.

♥ Indentación (sangría)

Los espacios al principio de una línea lógica marcan su **nivel de indentación**, y eso es lo que **agrupa las sentencias en bloques** (en C se usan llaves `{ }`; en Python, la sangría). Internamente se generan tokens **INDENT** y **DEDENT** usando una pila.

¡OJO!

No mezcles tabuladores y espacios para indentar. Consejo: usa siempre **espacios** (normalmente 4). VS Code lo hace solo al pulsar Tab.

ANALOGÍA

La indentación es como el esquema de unos apuntes: lo que está más metido hacia la derecha "pertenece" al título que tiene encima. En Python no es estética: si te equivocas de sangría, cambia el significado del programa (o da error).

♥ Tokens

Los espacios separan tokens, pero solo son **necesarios** si sin ellos dos tokens se leerían como uno (`x=3` y `x = 3` son lo mismo; `ifx` no es `if x`). Tipos de tokens: NEWLINE INDENT DEDENT identificadores palabras reservadas literales operadores delimitadores.

3.2 Identificadores

Son los **nombres** que pones a variables, funciones, clases y módulos.

- Pueden usar caracteres Unicode, pero por compatibilidad es mejor **solo ASCII** (sin tildes ni ñ).
- **No pueden empezar por un dígito**, ni contener espacios, signos de puntuación o delimitadores.
- Su longitud es ilimitada.
- **Distinguen mayúsculas y minúsculas**: `nota`, `Nota` y `NOTA` son tres nombres distintos.
- No pueden ser **palabras reservadas** (`if`, `for`, `def`, `True`, `None`, `and`…).

Válidos: `nota_final`, `_x`, `suma2`, `sumaCuadrados`. No válidos: `2suma`, `mi nota`, `for`, `nota-final`.

TRUCO · ESTILO

En Python se recomienda escribir los nombres de variables y funciones en **snake_case** (`suma_cuadrados`). En las transparencias verás también *camelCase* (`sumaCuadrados`); funciona igual, pero elige uno y sé coherente.

3.3 Literales

DEFINICIÓN

Literal: forma de escribir directamente un **valor constante** de un tipo predefinido. `42`, `3.14`, `"hola"` y `True` son literales.

♥ Cadenas (strings)

- **Cortas**: entre comillas simples o dobles, en una sola línea lógica: `'hola'`, `"hola"`.
- **Largas**: entre tres comillas (`"""…"""` o `'''…'''`), pueden ocupar varias líneas.
- Admiten cualquier carácter Unicode e interpretan el carácter de escape `\`.
- Hay prefijos: `r` (raw: sin escapes), `b` (bytes), `u` (unicode), `f` (con formato). Por ahora usamos cadenas sin prefijo, que en Python 3 ya son Unicode.

♥ Números

Tipo	Cómo se escriben	Ejemplos
Enteros	En decimal (sin empezar por 0) o con prefijo: `0b` binario, `0o` octal, `0x` hexadecimal. Se puede usar `_` para separar grupos. **¡No tienen límite de tamaño!** (solo la memoria)	`-1879`, `76_546_739_872`, `0xA35f`, `0B101010`
Reales (float)	Con punto decimal o con exponente. Son de 64 bits, doble precisión	`3.14`, `6.`, `.0001`, `3.14159e-6`, `0e0`
Imaginarios	Con sufijo `j` o `J`	`(3+4j)`, `8.j`, `10j`

El infinito no tiene literal propio: se escribe `float('inf')` (o `math.inf`).

3.4 Tipos

DEFINICIÓN

Tipo: un conjunto de **valores** junto con las **operaciones** que se pueden hacer con ellos, su significado y la forma en que se almacenan.

Categoría	Tipos
Texto	`str`
Numéricos	`int`, `float`, `complex`
Booleano	`bool` (literales `True`, `False`)
Secuencias	`list`, `tuple`, `range`
Nada	`NoneType` (literal `None`)
Más (capítulo 6)	`set`, `dict`…

- El tipo lo fija el **literal** que se asigna: `x = 3` → int; `x = 3.0` → float.
- Operar con tipos incompatibles da un **error en tiempo de ejecución** (`"2" + 3` → TypeError), aunque hay conversiones automáticas (`2.0 + 3` → 5.0).
- Puedes forzar el tipo con su nombre: `int("5")`, `float(3)`, `str(7)`. Esto se llama **casting**.

EJERCICIO · ADIVINA EL RESULTADO

`2+3` · `2.0 + 3` · `0x2 + 0x3` · `"2" + '3'` · `2 and 3` · `2 or 3` · `2 & 3`

SOLUCIÓN

Expresión	Resultado	Por qué
`2+3`	`5`	suma de enteros
`2.0 + 3`	`5.0`	int + float → float (conversión automática)
`0x2 + 0x3`	`5`	son enteros escritos en hexadecimal; el resultado se muestra en decimal
`"2" + '3'`	`'23'`	con cadenas, `+` **concatena**
`2 and 3`	`3`	`and` devuelve el primer valor falso o, si todos son verdaderos, **el último**
`2 or 3`	`2`	`or` devuelve **el primer valor verdadero**
`2 & 3`	`2`	AND **bit a bit**: 10 & 11 = 10 (2)

♥ Tipos numéricos en detalle

- **int**: precisión ilimitada. `hex()`, `oct()`, `bin()` pasan a esas bases (`bin(10)` → `'0b1010'`). `int()` convierte a entero, y con un segundo argumento lee en cualquier base hasta 36: `int('ff', 16)` → 255.
- **float**: doble precisión IEEE 754. `float()` convierte a real.
- **complex**: `-3 + 2.0j`. `complex(re, im)` lo construye.

3.5 Objetos y variables

En Python **todo es un objeto**: números, cadenas, booleanos, listas… Un objeto tiene un **tipo**, contiene **datos** y permite ciertas **operaciones**. Se guarda en memoria y se le puede dar uno o varios **nombres**: eso son las **variables**.

ANALOGÍA · ETIQUETAS, NO CAJAS

En Python una variable **no es una caja** donde metes un valor: es una **etiqueta con un nombre** que pegas a un objeto. `x = [1, 2]` pega la etiqueta *x* a una lista. Si luego haces `y = x`, pegas una segunda etiqueta *y* **a la misma lista** (no la copias).

El tipo de un objeto decide sus operaciones y, muy importante, si es:

INMUTABLE

No puede cambiar. Para "cambiarlo" se crea otro objeto. `int`, `float`, `bool`, `str`, `tuple`.

MUTABLE

Puede cambiar por dentro sin dejar de ser el mismo objeto. `list`, `set`, `dict`.

♥ Variables

- Sirven para **guardar y recuperar valores**, y se nombran con identificadores válidos.

- Python **no exige declarar el tipo**: solo gestiona el tipo del valor que guarda la variable (**tipado dinámico**). Una misma variable puede guardar primero un int y luego una cadena.
- En los lenguajes con tipado **estático** (como C) se distingue la declaración (`int x;`) de la inicialización. Python **obliga a dar un valor** a una variable antes de usarla; si no, da `NameError`.
- La computación se produce por el **cambio de valor de las variables**.

3.6 Operadores

DEFINICIÓN

Operador: símbolo que representa una operación bien definida para uno o varios tipos. Hay que saber **qué hace con cada tipo** (`+` suma números pero concatena cadenas) y sus **reglas de precedencia y asociatividad**.

```
+   -   *   **   /   //   %   @   <<   >>   &   |   ^   ~   :=   <   >   <=   >=   ==
                                                                                  !=
```

♥ Operadores y funciones numéricas

Operación	Resultado	Ejemplo
`x + y` , `x - y` , `x * y`	suma, resta, producto	
`x / y`	cociente **real** (siempre float)	`7 / 2` → `3.5`
`x // y`	cociente **entero** (redondea hacia abajo)	`7 // 2` → `3` ; `-7 // 2` → `-4`
`x % y`	**resto** de x // y	`7 % 3` → `1`
`x ** y` , `pow(x, y)`	potencia	`2 ** 10` → `1024`
`-x`	x negado	
`abs(x)`	valor absoluto (módulo si es complejo)	`abs(-3+4j)` → `5.0`
`round(x, n)`	x redondeado a n decimales	`round(3.14159, 2)` → `3.14`
`int(x)` , `float(x)`	convertir (int **trunca**)	`int(3.99)` → `3`
`complex(re, im)`	número complejo	`complex(1, 2)` → `(1+2j)`
`c.conjugate()`	conjugado de c	`(3+4j).conjugate()` → `(3-4j)`
`divmod(x, y)`	la tupla `(x // y, x % y)`	`divmod(17, 5)` → `(3, 2)`

TRUCO · // Y %

Son la división "del colegio": 17 caramelos entre 5 amigos → a cada uno le tocan `17 // 5 = 3` y sobran `17 % 5 = 2` . Con `% 2` sabes si un número es par (resto 0) o impar (resto 1); con `% 10` sacas la última cifra y con `// 10` la quitas.

♥ Booleanos y valor de verdad

El tipo `bool` tiene dos valores: `True` y `False` . Pero **cualquier objeto** de Python puede evaluarse como booleano:

SE EVALÚAN A FALSE

`None`, `False`, los ceros (`0`, `0.0`) y las secuencias y colecciones **vacías**: `''`, `()`, `[]`, `{}`, `set()`, `range(0)`.

SE EVALÚAN A TRUE

Todo lo demás. Ojo: `' '` (un espacio) y `[0]` (una lista con un cero) son verdaderos, porque no están vacíos.

Operación	Resultado	Notas
`x or y`	disyunción (O)	evaluación **corta**: si x es verdadero, ni mira y
`x and y`	conjunción (Y)	evaluación **corta**: si x es falso, ni mira y
`not x`	negación	siempre devuelve True o False

```
>>> bool(0 ** 0)          # 0**0 está definido como 1
True
>>> bool(0.0)
False
>>> type(3) == type(3.)   # los float llevan el punto decimal
False
```

♥ Comparaciones

Hay 8, todas con la **misma prioridad** y siempre **más alta que las booleanas** (por eso `a > 2 and b <= 3` no necesita paréntesis):

`<`	`<=`	`>`	`>=`	`==`	`!=`	`is`	`is not`
menor	menor o igual	mayor	mayor o igual	igual	distinto	mismo objeto	distinto objeto

Se pueden **encadenar**: `x < y <= z` significa `x < y and y <= z` (y también se evalúa de forma corta).

¡OJO! == FRENTE A IS

`==` compara **valores**; `is` compara si son **el mismo objeto** en memoria. `[] == []` es `True` (dos listas vacías valen lo mismo) pero `[] is []` es `False` (son dos listas distintas). Usa `is` casi solo para `None`: `if x is None:`.

♥ Delimitadores

DEFINICIÓN

Delimitador: uno o varios caracteres que **separan o agrupan** otros tokens sin formar parte de ellos.

```
(   )   [   ]   {   }   ,   :   .   ;   @   =   ->   +=   -=   *=   /=   //=   %=   @=
                &=   |=   ^=   >>=   <<=   **=
```

Fíjate en que `=` y los operadores de asignación aumentada (`+=`…) son **delimitadores**, no operadores: la asignación es una sentencia, no una operación que devuelve un valor.

3.7 Cadenas: lo básico

```
>>> 'Hola Mundo'
'Hola Mundo'
>>> "'Sí', Hola Mundo"            # comillas simples dentro de dobles: sin problema
"'Sí', Hola Mundo"
>>> '\'Sí\', Hola Mundo'          # o escapándolas con \
"'Sí', Hola Mundo"
>>> print("Primera línea\nSegunda línea")    # \n = salto de línea
Primera línea
Segunda línea
>>> 3 * 'o' + 'h'                 # * repite, + concatena
'oooh'
>>> print('\u25B2')               # carácter Unicode por su code point
▲
```

- Una cadena **corta** no puede ocupar dos líneas: da `SyntaxError`. Para eso usa una cadena **larga** (`"""…"""`), o escapa el fin de línea con `\` (entonces las dos líneas se pegan: `'Primera líneaSegunda línea'`).
- Escapes útiles: `\n` salto de línea, `\t` tabulador, `\\` barra invertida, `\'` y `\"` comillas, `\uXXXX` carácter Unicode.

¡OJO CON LAS RUTAS DE WINDOWS!

`print("c:\home\nombreusuario")` imprime `c:\home` y en la línea siguiente `ombreusuario`, porque `\n` se interpreta como salto de línea. (Además, las versiones modernas de Python avisan de que `\h` no es un escape válido.) Soluciones: `"c:\\home\\nombreusuario"` o una cadena *raw*: `r"c:\home\nombreusuario"`.

¡OJO CON LA CODIFICACIÓN DE LOS FICHEROS!

Unicode asigna un número a cada carácter, pero para guardarlo en un fichero hace falta una **codificación concreta**. Lo habitual es UTF-8, pero Windows a veces usa CP-1252. Si mueves un `.py` con tildes o ñ entre sistemas, pueden salir caracteres raros. Guarda siempre en UTF-8.

3.8 Expresiones y sentencias

DEFINICIÓN

Expresión: combinación bien escrita de literales, variables, operadores, delimitadores y llamadas a funciones que, según las reglas de precedencia y asociación, **produce un valor** de cierto tipo (puede ser `None`).

- Toda expresión devuelve un valor. Si dudas de la precedencia, **usa paréntesis**.
- Se evalúan de **izquierda a derecha**.
- Pueden tener **efectos colaterales** además de calcular un valor (por ejemplo, escribir en pantalla).
- Ejemplos: `7*x+5` , `- -x` , `b and (a or b)` , `-3/sin(x)` .

DEFINICIÓN

Sentencia: la unidad fundamental de ejecución. Debe ser sintácticamente correcta, se ejecutan **secuencialmente** (algunas provocan saltos) y pueden contener expresiones. En Python, una expresión sola también es una sentencia.

♥ La sentencia de asignación

```
x = x + 1      # valor_izq  operador_de_asignación  valor_der
```

¡ESTO NO ES MATEMÁTICAS!

En mates, *x = x + 1* es imposible. En programación `=` significa "**calcula lo de la derecha y guárdalo en lo de la izquierda**". Si x valía 5, después vale 6. Para **comparar** se usa `==`.

PRECEDENCIA, DE MÁS A MENOS (LO ESENCIAL)

`()` → `**` → `-x` → `* / // %` → `+ -` → comparaciones (`< == in is`…) → `not` → `and` → `or`. Así, `A and not B or C` significa `(A and (not B)) or C`.

4 Sentencias de control

4.1 Problemas, algoritmos y programas

Concepto	Definición
Problema	Un asunto del que se espera encontrar una solución.
Algoritmo	Conjunto **finito** de reglas **precisas**, en términos de operaciones elementales, que permiten hacer una tarea en un **tiempo finito**.
Programa	Artefacto construido por una persona, escrito en un lenguaje de programación y destinado a ser ejecutado por un ordenador.
Problema decidible	El que puede resolverse con un algoritmo. Su solución se llama **constructiva**.

No todo se puede resolver con un algoritmo. Ejemplos de problemas **no decidibles**:

- **El problema de decisión**: saber si una fórmula de lógica de primer orden es un teorema.
- **El problema de la parada**: saber si un programa cualquiera terminará en un tiempo finito.
- **El problema del embaldosado**: cubrir el plano con un conjunto dado de baldosas cuadradas de lados de colores, de forma que los lados que se tocan tengan el mismo color.

♥ Tesis de Church-Turing

El concepto de algoritmo es **intuitivo**, no se puede definir matemáticamente. Para precisar qué es "computable" se usa un modelo abstracto: la **máquina de Turing**. Hay muchos modelos de computación, pero los fundamentales son todos **equivalentes** a ella.

TESIS DE CHURCH-TURING

La noción de algoritmo coincide con la noción de cómputo en una máquina de Turing.

4.2 Programación estructurada

DEFINICIONES

Programa propio: tiene **un único punto de entrada y un único punto de salida**, y toda sentencia se ejecuta para algún dato de entrada (no hay código "muerto").

Programa estructurado: se puede escribir usando **solo** sentencias de **secuencia**, **selección** y **repetición**.

TEOREMA DE LA PROGRAMACIÓN ESTRUCTURADA

Todo programa propio es equivalente a un programa estructurado.

Secuencia	**Selección**	**Repetición**
una tras otra	if	for, while

ANALOGÍA

Con tres piezas de Lego (secuencia, selección y repetición) puedes construir cualquier programa. No necesitas "saltos" a lo loco (los `goto` del ensamblador).

4.3 Sentencias simples y compuestas

Las sentencias se ejecutan **de arriba abajo**. Ese flujo se rompe con las sentencias de **selección** (`if`), **repetición** (`for`, `while`) y **bifurcación** (`break`, `continue`).

♥ Simples (ocupan una línea lógica)

Se pueden poner varias en una línea separándolas con `;` (aunque no se recomienda).

- **De expresión**: calculan un valor o llaman a una función (sobre todo en modo interactivo).
- **De asignación**: ligan nombres a valores. Se pueden asignar varios a la vez:

```
mi_var = 33
producto, i = 1, 10       # asignación múltiple (con tuplas): producto = 1, i = 10
a, b = b, a               # ¡intercambiar dos variables en una línea!
mi_var += 1               # asignación aumentada: mi_var = mi_var + 1
```

Asignaciones aumentadas: `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`… Internamente no son 100 % iguales a la versión larga (con listas, `+=` modifica la lista original), pero para números sí.

♥ Compuestas

Contienen otros bloques de sentencias y controlan su ejecución: `if`, `for`, `while` (y más adelante `def`, `try`…). Llevan siempre **dos puntos** `:` al final de la cabecera y el bloque **indentado** debajo.

4.4 La sentencia if

if

```
if expresion:
    bloque
```

Si la expresión es verdadera, se ejecuta el bloque; si no, se salta. Después se sigue detrás.

if – else

```
if expresion:
    bloque1
else:
    bloque2
```

Se ejecuta **uno y solo uno** de los dos bloques.

if – elif – else

```
if exp1:
    bloque1
elif exp2:
    bloque2
else:
    bloque3
```

Se ejecuta el bloque de la **primera** expresión verdadera; si ninguna lo es, el del `else`.

- La expresión puede ser **cualquiera**: se interpreta por su valor de verdad (apartado 3.6).
- `elif` es una abreviatura de *else if*. Puede haber tantos como quieras; `elif` y `else` son **opcionales**.
- Dentro de un bloque la indentación debe ser la misma, aunque puede variar entre bloques distintos (pero mejor usa siempre 4 espacios).

EJEMPLO · POSITIVO, NEGATIVO O CERO

```
x = int(input('Introduzca un entero: '))
if x < 0:
    print("Ha introducido un número negativo")
elif x == 0:
    print("Ha introducido el cero")
else:
    print("Ha introducido un número positivo")
```

EJEMPLO · DE NOTA NUMÉRICA A NOTA EN TEXTO

En la universidad: [0, 5) suspenso · [5, 7) aprobado · [7, 9) notable · [9, 9.5) sobresaliente · [9.5, 10] matrícula de honor.

```
nota = float(input("Introducir la nota a convertir: "))
if nota < 0 or nota > 10:
    print("Nota inválida")
elif nota < 5:
    print("Suspenso")
elif nota < 7:          # si llegamos aquí, ya sabemos que nota >= 5
    print("Aprobado")
elif nota < 9:
    print("Notable")
elif nota < 9.5:
    print("Sobresaliente")
else:
    print("Matrícula de Honor")
```

Con 4.7 imprime *Suspenso*. En el cuaderno se escribe `elif 5 <= nota < 7`, que también funciona (comparación encadenada), pero no hace falta repetir el límite inferior: cada `elif` solo se mira si los anteriores fueron falsos.

¡OJO! CORRECCIÓN

En la versión del cuaderno, una nota negativa (por ejemplo -3) imprime "Suspenso" en vez de "Nota inválida", porque la primera condición `nota < 5` ya es verdadera. Por eso compruebo primero que la nota sea válida.

♥ if anidados: cuidado con la indentación

Estos dos programas de las transparencias solo se diferencian en la sangría, y hacen cosas distintas:

```
if a > 2 and b <= 3:
    if not par:
        c = 10
    if a == 3:
        z = 33
    else:           # ← es del "if a == 3"
        z = 31
else:
    z = 0
```

```
if a > 2 and b <= 3:
    if not par:
        c = 10
        if a == 3:
            z = 33
    else:           # ← es del "if not par"
        z = 31
else:
    z = 0
```

Con `a = 4, b = 1, par = False`: en el de la izquierda, `a == 3` es falso → **z = 31**. En el de la derecha, `not par` es verdadero, se entra, `a == 3` es falso y no se hace nada más → **z no se asigna**. Un `else` pertenece al `if` que tiene **exactamente su misma sangría**.

(Los `;` al final de línea que aparecen en las transparencias no hacen falta en Python; los he quitado.)

CÁLCULO BOOLEANO

Para simplificar condiciones conviene saber las **leyes de De Morgan**: `not (A and B)` = `not A or not B` · `not (A or B)` = `not A and not B`. Y las **distributivas**: `A and (B or C)` = `(A and B) or (A and C)`.

4.5 La sentencia for

```
for x in secuencia:
    bloque
```

Repite el bloque **una vez por cada elemento** de la secuencia; en cada vuelta, `x` toma el valor del siguiente elemento. La secuencia puede ser cualquier estructura recorrible: `range`, una cadena, una lista, una tupla…

♥ La función range()

Forma	Genera	Ejemplo
`range(fin)`	0, 1, …, fin-1	`range(5)` → 0 1 2 3 4
`range(inicio, fin)`	inicio, …, fin-1	`range(3, 6)` → 3 4 5
`range(inicio, fin, paso)`	de paso en paso, **sin llegar a fin**	`range(10, 21, 2)` → 10 12 … 20
paso negativo	cuenta hacia atrás	`range(10, 0, -3)` → 10 7 4 1

TRUCO

El **fin nunca se incluye**. Por eso para ir de 1 a n se escribe `range(1, n + 1)`. Puedes ver lo que genera con `list(range(10))` → `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`.

EJEMPLOS CON FOR

```
# Imprimir los 10 primeros naturales
for j in range(10):
    print(j + 1)

# Sumar los 10 primeros naturales
suma = 0
for n in range(10):
    suma += n + 1
print(suma)                    55
```

```
# Pares entre 10 y 20 y su producto
producto = 1
for k in range(10, 21, 2):
    producto *= k
    print(k)
print("El producto es:", producto)
10 12 14 16 18 20 (uno por línea)
El producto es: 9676800
```

`suma += n + 1` **acumula**: equivale a `suma = suma + n + 1`. Patrón del **acumulador**: empieza en 0 para sumar, en 1 para multiplicar.

¡OJO! NO LLAMES A UNA VARIABLE SUM

En el cuaderno se usa `sum = 0`. Funciona, pero **tapa** la función predefinida `sum()`, y si luego quieres usarla (`sum([1, 2])`) dará error. Usa `suma`, `total`… Lo mismo con `list`, `str`, `max`…

4.6 La sentencia while

```
while condicion:
    bloque
```

Ejecuta el bloque **mientras** la condición sea verdadera. Se comprueba **antes** de cada vuelta: si al principio ya es falsa, el bloque no se ejecuta nunca.

```
# Los 10 primeros naturales con while
i = 1
while i <= 10:
    print(i)
    i += 1      # ¡sin esto, bucle infinito!
```

```
# Pares de 10 a 20 y su producto
producto, i = 1, 10
while i <= 20:
    producto *= i
    print(i)
    i += 2
print("El producto es:", producto)
```

¿FOR O WHILE?

for: cuando sabes **cuántas veces** repetir o tienes una secuencia que recorrer. **while**: cuando repites **hasta que pase algo** (el usuario acierta, llega una línea vacía…).

EJEMPLO · SUMAR NÚMEROS HASTA UNA LÍNEA VACÍA

```
linea = input("Introducir números enteros, uno por línea, yo te los sumaré:\n")
suma = 0
while linea != "":
    suma += int(linea)
    linea = input()
print("La suma es:", suma)       # con 3, 4, 5, 6 y Enter → La suma es: 18
```

♥ break y continue

- `break`: **sale del bucle** inmediatamente.
- `continue`: **salta al principio de la siguiente vuelta**, saltándose el resto del bloque.

```
for n in range(1, 10):
    if n % 2 == 0:
        continue        # los pares no se imprimen
    if n > 7:
        break           # al llegar a 9 se acaba el bucle
    print(n)            1 3 5 7
```

♥ Recorrer cadenas, tuplas y listas

Las cadenas, tuplas y listas son **secuencias** y se recorren igual con `for`. Las cadenas y tuplas son **inmutables**; las listas, **mutables** (lo verás a fondo en el capítulo 6):

```
cadena = 'hola'; tupla = ('h', 'o', 'l', 'a'); lista = ['h', 'o', 'l', 'a']
for c in cadena:      # igual con tupla o con lista
    print(c)           h  o  l  a (uno por línea)
lista[0] = 'H'         # bien: las listas son mutables
cadena[0] = 'H'        TypeError: 'str' object does not support item assignment
```

4.7 Problemas del cuaderno 03 (resueltos)

PROBLEMA · INVERTIR LOS NÚMEROS DE LA ENTRADA ESTÁNDAR

Leer números (uno por línea, hasta una línea vacía) y mostrarlos en orden inverso.

SOLUCIÓN

```
numeros = []
linea = input()
while linea != "":
    numeros.append(int(linea))
    linea = input()
for i in range(len(numeros) - 1, -1, -1):   # de la última posición a la 0
    print(numeros[i])
```

Con 3, 4, 5 imprime 5, 4, 3. Versión corta: `for n in reversed(numeros): print(n)`.

Si lo que se pedía era invertir **las cifras** de un número (1234 → 4321), se hace con `%` y `//`: `inv = 0` y mientras `n > 0`: `inv = inv * 10 + n % 10`; `n //= 10`.

PROBLEMA · INVERTIR UNA CADENA LEÍDA EN UNA LÍNEA

SOLUCIÓN

```
# Con un bucle
texto = input()
invertida = ""
for c in texto:
    invertida = c + invertida   # cada let
print(invertida)
```

```
# Con slicing (capítulo 6)
texto = input()
print(texto[::-1])

# "hola" → "aloh"
```

4.8 Hoja de repaso: condicionales y bucles (resuelta)

EJERCICIO 1 · POSITIVO, NEGATIVO O CERO

```
numero = float(input("Ingrese un número: "))
if numero > 0:
    print("El número es positivo.")
elif numero < 0:
    print("El número es negativo.")
else:
    print("El número es cero.")
```

EJERCICIO 2 · EL MAYOR DE TRES NÚMEROS

```
a = float(input("Ingrese el primer número: "))
b = float(input("Ingrese el segundo número: "))
c = float(input("Ingrese el tercer número: "))
mayor = a
if b > mayor:
    mayor = b
if c > mayor:
    mayor = c
print("El número mayor es:", mayor)       # o simplemente max(a, b, c)
```

Si quieres que con 12 imprima `12` y no `12.0`, lee los números con `int()`.

EJERCICIO 3 · PAR O IMPAR

```
n = int(input("Ingrese un número entero: "))
if n % 2 == 0:
    print("El número es par.")
else:
    print("El número es impar.")
```

EJERCICIO 4 · ADIVINAR EL NÚMERO (WHILE)

```
import random
secreto = random.randint(1, 10)      # entero aleatorio entre 1 y 10, ambos incluidos
intento = int(input("Adivina el número (1-10): "))
while intento != secreto:
    if intento < secreto:
        print("El número es mayor.")
    else:
        print("El número es menor.")
    intento = int(input("Prueba otra vez: "))
print("¡Correcto! Era el", secreto)
```

EJERCICIO 5 · PARES DEL 1 AL 20 (FOR)

```
for n in range(2, 21, 2):
    print(n, end=" ")                2 4 6 8 10 12 14 16 18 20
```

EJERCICIO 6 · SUMAR DEL 1 AL 100 (WHILE)

```
suma, i = 0, 1
while i <= 100:
    suma += i
    i += 1
print("La suma de los números del 1 al 100 es:", suma)     5050
```

EJERCICIO 7 · CONTAR NEGATIVOS ENTRE 5 NÚMEROS

```
negativos = 0
for i in range(1, 6):
    n = float(input(f"Ingrese el número {i}: "))
    if n < 0:
        negativos += 1
print("Cantidad de números negativos:", negativos)       # -5 3 -2 0 8 → 2
```

`f"…{i}…"` es una **f-string**: mete el valor de `i` dentro del texto.

EJERCICIO 8 · FACTORIAL CON WHILE

```
n = int(input("Ingrese un número entero positivo: "))
factorial, i = 1, n
while i > 1:
    factorial *= i
    i -= 1
print(f"El factorial de {n} es: {factorial}")          5 → 120
```

EJERCICIO 9 · MULTIPLICAR POR 2 CADA NÚMERO DE UNA LISTA

```
lista = [2, 5, 10]
for n in lista:
    print(n * 2, end=" ")              4 10 20
```

EJERCICIO 10 · CONTAR VOCALES

```
cadena = input("Ingrese una cadena: ")
vocales = 0
for c in cadena.lower():           # lower() para contar también A, E…
    if c in "aeiouáéíóú":
        vocales += 1
print("Cantidad de vocales:", vocales)       # "Hola Mundo" → 4
```

EJERCICIO 11 · MEDIA, MODA Y MEDIANA

```
import statistics
texto = input("Ingrese una lista de números separados por comas: ")
numeros = [int(x) for x in texto.split(",")]   # "1, 2, 2" → [1, 2, 2]
print("Media:", float(statistics.mean(numeros)))
print("Moda:", statistics.mode(numeros))
print("Mediana:", statistics.median(numeros))
Media: 4.0
Moda: 2
Mediana: 3
```

`split(",")` corta la cadena por las comas, e `int()` ignora los espacios de alrededor. La **media** es la suma entre la cantidad; la **moda**, el valor que más se repite; la **mediana**, el valor central una vez ordenados.

5 Funciones

5.1 Usar y definir funciones

Ya has usado funciones **predefinidas**: `print()`, `len()`, `input()`… Para usarlas escribes su nombre, paréntesis y los argumentos:

```
lista = [3, 4, 5]
x = len(lista)       # len() devuelve 3
print(lista)         # print() escribe en pantalla, pero devuelve None
```

Y también puedes **definir las tuyas** con la palabra reservada `def`:

```
def nombre_funcion(param1, param2):       # cabecera: nombre y parámetros formales
    """Docstring opcional: qué hace la función."""
    cuerpo                                # indentado
    return resultado
```

- El nombre debe ser un **identificador válido**.
- Los **parámetros formales** van entre paréntesis, separados por comas. Aunque no haya ninguno, **los paréntesis son obligatorios** (al definir y al llamar).
- La primera línea del cuerpo puede ser una cadena de documentación (**docstring**).
- `return expresión` **termina** la función y devuelve el valor de la expresión. Puede haber 0, 1 o varios `return` (una función puede salir por varios puntos).

EJEMPLO

```
def cuadrado(n):
    return n ** 2

x = cuadrado(100)      # x = 10000
```

ANALOGÍA · UNA MÁQUINA DE ZUMOS

Una función es una **máquina**: le metes cosas por la entrada (los **argumentos**), hace su trabajo por dentro (el **cuerpo**) y te devuelve un resultado (el **return**). Para usarla no necesitas saber cómo funciona por dentro, solo qué meter y qué sale.

5.2 Llamadas a funciones

Una llamada `nombre_funcion(argumentos)` forma parte de una expresión. La función se ejecuta y **el valor que devuelve sustituye a la llamada**; luego se sigue con lo que viniera detrás.

- Una función que **no devuelve** nada se suele llamar **procedimiento** o **subrutina**. En Python, una función sin `return` devuelve `None` (como `print()`).
- Tiene que haber un **emparejamiento** correcto entre los **parámetros formales** (los de la definición) y los **parámetros reales o argumentos** (los de la llamada).

¿PARA QUÉ SIRVEN LAS FUNCIONES?

Evitan repetir código. · **Encapsulación**: solo te interesa la entrada y la salida, no cómo se calcula ni qué memoria usa. · **Abstracción**: eliminas detalles para centrarte en lo que hace, y así puedes construir programas más complejos.

5.3 La pila de llamadas

EJEMPLO · FUNCIONES SIN VALOR DE RETORNO

```
def nueva_linea():
    'Escribe en stdout una línea en blanco'
    print()

def tres_lineas():
    'Escribe en stdout 3 líneas en blanco'
    nueva_linea()
    nueva_linea()
    nueva_linea()

def main():
    print("Linea 1")
    tres_lineas()
    print("Linea 2")

main()
```

Imprime "Linea 1", tres líneas en blanco y "Linea 2". Las tres primeras partes solo **definen** funciones; lo único que se ejecuta de verdad es la última línea, `main()`.

¿Cómo sabe Python a dónde volver cuando una función termina? Usa una **pila de llamadas**. Cada vez que se llama a una función se apila un **marco** con la **dirección de retorno**, los **argumentos** y el sitio para el **resultado**. Al terminar, se desapila y se vuelve a donde se quedó.

Momento	Pila (la cima a la derecha)
Se ejecuta `main()`	main
`print("Linea 1")`	main → print
Termina print; se llama a `tres_lineas()`	main → tres_lineas
Primera `nueva_linea()`	main → tres_lineas → nueva_linea
Dentro, `print()`	main → tres_lineas → nueva_linea → print
print termina, nueva_linea termina	main → tres_lineas
(lo mismo con la 2ª y 3ª nueva_linea)	…
tres_lineas termina; `print("Linea 2")`	main → print
Todo termina	(vacía)

ANALOGÍA

Es como leer un libro con notas al pie que remiten a otras notas: cada vez que saltas a una nota, dejas un **marcapáginas** encima del montón. Al acabar una nota, coges el marcapáginas de arriba y vuelves allí. El último que pusiste es el primero que quitas (pila).

EJEMPLO · FUNCIONES CON VALOR DE RETORNO

```
def cuadrado(n):
    """Calcula el cuadrado de un número."""
    return n * n

def suma_cuadrados(n):
    """Calcula la suma de los cuadrados desde 1 hasta n."""
    resultado = 0
    for i in range(1, n + 1):
        resultado += cuadrado(i)
    return resultado

def main():
    leido = int(input())
    print("La suma de los", leido,          # se puede partir una línea entre argumentos
          "primeros cuadrados es", suma_cuadrados(leido))

main()       # con 4 → La suma de los 4 primeros cuadrados es 30
```

5.4 Tipos de parámetros

♥ 1) Posicionales

Los argumentos se emparejan con los parámetros **por orden**. Tiene que haber el mismo número. Los parámetros **no declaran tipo**, así que si los argumentos no sirven, el error aparece **al ejecutar**:

```
def fon(p1, p2, p3):
    print('El doble del parámetro 1 vale:', 2 * p1)
    print('El primer carácter del parámetro 2 vale:', p2[0])
    print('La longitud del parámetro 3 es:', len(p3))

fon(10, 20, 30)             El doble del parámetro 1 vale: 20
TypeError: 'int' object is not subscriptable    ← 20[0] no tiene sentido
fon(10, "hola", [1, 2, 3])  20 · h · 3   ← así sí funciona
```

♥ 2) Con valores por defecto

Si en la llamada faltan argumentos, se usa el valor por defecto:

```
def son_iguales(f1, f2, epsilon=1.0e-2):
    return abs(f1 - f2) < epsilon

son_iguales(0.999, 1.0)           True
son_iguales(0.999, 1.0, 1.0e-3)   False
print(1.0 - 0.999)                0.0010000000000000009
```

¿Por qué el segundo da False? Porque `1.0 - 0.999` no da exactamente 0.001, sino un poquito más (por la representación binaria de los reales), y eso **no es menor** que 0.001. Por eso los reales se comparan "con margen" (epsilon) y no con `==`.

- Una función así se puede llamar de varias formas.
- Los valores por defecto se calculan **al definir** la función, no al llamarla.
- Después de un parámetro con valor por defecto **no puede ir** uno sin él.

¡OJO! NO USES UNA LISTA COMO VALOR POR DEFECTO

Como el valor por defecto se crea **una sola vez**, una lista por defecto se comparte entre llamadas:

```
def f(valor, lista=[]):
    lista.append(valor)
    return lista
f(1)    [1]
f(2)    [1, 2]   ← ¡sorpresa!
```

```
def g(valor, lista=None):
    if lista is None:
        lista = []
    lista.append(valor)
    return lista       # g(1) [1] · g(2) [2]
```

EJEMPLO DE LA HOJA DE FUNCIONES

```
def saludar(nombre, saludo="Hola"):
    print(saludo + ", " + nombre)
saludar("Carlos")                 Hola, Carlos
saludar("Ana", "Buenos días")     Buenos días, Ana
```

♥ 3) Argumentos por nombre

En la llamada puedes escribir `parametro=valor`, y entonces el orden da igual:

```
def sol_segundo_grado(c, b, a=1):
    """Devuelve las soluciones de la ecuación c + b*x + a*x**2 = 0"""
    x1 = -b / (2*a)
    x2 = (b**2 - 4*a*c) ** (1/2) / (2*a)
    return (x1 + x2), (x1 - x2)

sol_segundo_grado(-1, 0)              (1.0, -1.0)     ← x² − 1 = 0
sol_segundo_grado(a=1, b=0, c=-1)     (1.0, -1.0)
sol_segundo_grado(-1, 0, c=1)         TypeError: ... got multiple values for argument 'c'
```

- Es la fórmula de siempre, $x = (-b \pm \sqrt{(b^2-4ac)}) / 2a$, partida en dos trozos. `** (1/2)` es la raíz cuadrada.
- Después de un argumento con nombre **no puede ir** uno sin nombre.
- Un parámetro **nunca puede recibir dos valores**: en la última llamada, `c` recibe -1 por posición y 1 por nombre → error.

♥ 4) Número variable de argumentos: *args

Si el último parámetro lleva **un asterisco** (`*params`), todos los argumentos sobrantes se **empaquetan en una tupla** con ese nombre. Después de él, solo se pueden pasar argumentos por nombre.

```
def suma_todo(*numeros):           # numeros es una tupla
    return sum(numeros)
suma_todo(1, 2, 3)                   6

def presentar(nombre, *aficiones, ciudad="León"):
    print(nombre, aficiones, ciudad)
presentar("Lucía", "leer", "gym")              Lucía ('leer', 'gym') León
presentar("Lucía", "leer", ciudad="Madrid")    Lucía ('leer',) Madrid
```

♥ 5) Argumentos con nombre ilimitados: **kwargs

Con **dos asteriscos** (`**keywords`, al final de todo), los argumentos con nombre que sobran se empaquetan en un **diccionario** `{nombre_parámetro: valor}`:

```
def ficha(**datos):
    for clave, valor in datos.items():
        print(clave, "=", valor)
ficha(nombre="Lucía", edad=18)       nombre = Lucía
edad = 18
```

♥ 6) Solo posicionales (/) y solo por nombre (*)

```
def f(pos1, pos2, /, pos_o_clave, *, clave1, clave2):
```

Zona	Cómo se pueden pasar
Antes de `/`	**Solo por posición**
Entre `/` y `*`	Por posición **o** por nombre
Después de `*`	**Solo por nombre**
Sin `/` ni `*`	Todos de las dos formas

```
def fpos(a, b, /, c, *, d):
    return a + b + c + d
fpos(1, 2, 3, d=4)         10
fpos(1, 2, c=3, d=4)       10
fpos(a=1, b=2, c=3, d=4)   TypeError (a y b solo admiten posición)
```

ORDEN EN LA CABECERA

normales → con valor por defecto → `*args` → solo por nombre → `**kwargs`.

5.5 Devolver varios valores

Una función puede devolver varios valores separados por comas: en realidad devuelve una **tupla**, que puedes desempaquetar:

```python
def calculos(n):
    cuadrado = n * n
    cubo = n * n * n
    return cuadrado, cubo

resultado_cuadrado, resultado_cubo = calculos(3)
print("Cuadrado:", resultado_cuadrado)     Cuadrado: 9
print("Cubo:", resultado_cubo)             Cubo: 27
calculos(3)                                (9, 27)
```

5.6 Documentar y anotar tipos

♥ Docstrings

```python
def area_rectangulo(base, altura):
    """
    Calcula el área de un rectángulo.

    Parámetros:
    base -- la base del rectángulo (int o float)
    altura -- la altura del rectángulo (int o float)

    Retorna:
    El área del rectángulo.
    """
    return base * altura

print(area_rectangulo(5, 3))       15
help(area_rectangulo)              # muestra el docstring
```

♥ Anotaciones de tipo (type hints)

Desde Python 3.5 se puede indicar el tipo de los parámetros y del resultado:

```python
def sumar(a: int, b: int) -> int:
    return a + b
print(sumar(4, 5))          9
print(sumar("ho", "la"))    hola   ← ¡no da error!
```

Las anotaciones **no se comprueban al ejecutar**: son documentación que ayuda a leer el código y a las herramientas de análisis (como VS Code) a avisarte de errores.

5.7 Funciones lambda

Son funciones **anónimas** de una sola expresión, sin `def` ni `return`:

```python
multiplicar = lambda x, y: x * y
print(multiplicar(3, 4))                                  12
sorted(['banana', 'Kiwi', 'apple'], key=lambda s: s.lower())  ['apple', 'banana', 'Kiwi']
list(map(lambda x: x * 2, [1, 2, 3]))                     [2, 4, 6]
list(filter(lambda x: x % 2 == 0, range(10)))             [0, 2, 4, 6, 8]
```

Son útiles como argumento de otras funciones: `map()` aplica la función a cada elemento, `filter()` se queda con los que la cumplen y `sorted(…, key=…)` ordena según lo que devuelva.

5.8 Ámbito de las variables (tablas de símbolos)

- Cada identificador (variable, función…) se guarda en una **tabla de símbolos** del "nivel" donde se define, y tiene un **ámbito** (*scope*) donde se puede usar.
- Al ejecutar una función se crea una **tabla local nueva** con sus variables locales y sus parámetros. **Se borra al terminar** la función.
- Cuando usas un nombre, Python lo busca en este orden: **1)** tabla local, **2)** tablas de las funciones que la contienen, **3)** tabla **global**, **4)** nombres **predefinidos** (`print`, `len`…).
- Si dentro de una función **asignas** a una variable con el mismo nombre que una global, en realidad **creas una variable local nueva**. Para modificar la de fuera hay que declararlo con `global` (la del módulo) o `nonlocal` (la de la función que la contiene).

ANALOGÍA

Las tablas de símbolos son como **habitaciones**. Cuando buscas algo, primero miras en tu cuarto (local); si no está, en el salón de la casa (función contenedora), luego en el portal del edificio (global) y por último en la calle (predefinidos). Lo que dejas en tu cuarto desaparece al salir de él.

EJEMPLO · GLOBAL

```
x = "global"
def fx():
    x = "local en fx"     # x NUEVA, local
    print("x en fx:", x)

fx()
print("x global:", x)
x en fx: local en fx
x global: global
```

```
y = "global"
def gy():
    global y              # usa la y de fuera
    y = "local en gy"
    print("y en gy:", y)

gy()
print("y global:", y)
y en gy: local en gy
y global: local en gy
```

EJEMPLO · NONLOCAL (FUNCIONES DENTRO DE FUNCIONES)

```
x = "global"
def fx():
    x = "local de fx"
    def sx():
        x = "local de sx"
        print("x en sx:", x)
    sx()
    print("x en fx:", x)
fx()
print("x global:", x)
x en sx: local de sx
x en fx: local de fx
x global: global
```

```
y = "global"
def fy():
    y = "local de fy"
    def sy():
        nonlocal y        # la y de fy
        y = "local de sy"
        print("y en sy:", y)
    sy()
    print("y en fy:", y)
fy()
print("y global:", y)
y en sy: local de sy
y en fy: local de sy
y global: global
```

`nonlocal` modifica la variable de la función de fuera (`fy`), pero no la global.

¡OJO! EL ERROR TÍPICO

```
contador = 0
def incrementar():
    contador += 1      UnboundLocalError: cannot access local variable 'contador'
```

Como hay una asignación, Python decide que `contador` es local… y todavía no tiene valor. Solución: `global contador` (o, mejor, pasar el valor como parámetro y devolverlo).

5.9 ¿Cómo se pasan los argumentos?

Dicho con precisión: en Python los argumentos se pasan **por valor de la referencia al objeto**. La función recibe una nueva etiqueta pegada **al mismo objeto**:

- Si dentro **reasignas** el parámetro (`l = [0]`), solo mueves la etiqueta local: **fuera no cambia nada**.
- Si el objeto es **mutable** y lo **modificas** (`l.append(3)`), el cambio **se ve fuera**, porque es el mismo objeto.

```
unalista = [1, 2]
print(hex(id(unalista)))       # id() = identificador (dirección) del objeto

def f(l):
    l.append(3)                # modifica EL MISMO objeto → se ve fuera
    print(hex(id(l)))          # misma dirección que unalista
    l = [0]                    # l apunta ahora a otra lista (local)
    print(hex(id(l)))          # dirección distinta

f(unalista)
print(hex(id(unalista)))       # la misma de antes
print(unalista)                [1, 2, 3]
```

ANALOGÍA

Le das a una amiga una **copia de la llave** de tu casa (la referencia). Si entra y mueve los muebles (`append`), al volver tú los verás movidos. Si ella decide usar su llave para otra casa (`l = [0]`), tu casa no cambia.

Con objetos **inmutables** (números, cadenas, tuplas) no hay forma de modificarlos, así que en la práctica se comportan como un "paso por valor": lo de fuera nunca cambia.

5.10 Funciones recursivas

DEFINICIÓN

Una función es **recursiva** si se llama a sí misma dentro de su cuerpo. Crea repeticiones como un bucle, y se detiene porque hay **casos base** en los que ya no se llama a sí misma.

```python
def factorial(n):
    """Calcula el factorial del número natural n."""
    if n == 0:                    # caso base
        return 1
    return n * factorial(n - 1)     # caso recursivo: un problema más pequeño
```

Bajando (se apilan llamadas)	Subiendo (se resuelven)
factorial(3) = 3 · factorial(2)	= 3 · 2 = **6**
factorial(2) = 2 · factorial(1)	= 2 · 1 = 2
factorial(1) = 1 · factorial(0)	= 1 · 1 = 1
factorial(0) = 1 ← caso base	1

ANALOGÍA · LAS MUÑECAS RUSAS

Para abrir una matrioska abres la de fuera y te encuentras otra igual pero más pequeña; repites hasta llegar a la diminuta que no se abre (**caso base**). Si no hubiera muñeca final, nunca terminarías.

RECETA PARA PENSAR EN RECURSIVO

1) ¿Cuál es el caso más simple, que sé resolver directamente? (caso base). **2)** Si supiera resolver el problema para n-1, ¿cómo resuelvo el de n? (caso recursivo). **3)** Comprueba que cada llamada se acerca al caso base.

¡OJO!

Sin caso base (o si nunca se alcanza, por ejemplo `factorial(-1)`), la función se llama infinitamente hasta que Python corta con `RecursionError` (el límite son unas 1000 llamadas apiladas).

5.11 La función main y __name__

Python **no obliga** a tener una función `main()` (a diferencia de C o Java), pero es buena práctica en programas grandes: mejora la organización y separa "ejecutar el script" de "usarlo como módulo" en otro programa.

```python
def main():
    print("Este es el inicio del programa.")

if __name__ == "__main__":
    main()
```

Cada fichero tiene una variable especial `__name__`. Si el fichero **se ejecuta directamente**, vale `"__main__"`; si se **importa** desde otro (`import mi_fichero`), vale el nombre del módulo. Así, `main()` solo se ejecuta cuando lanzas el fichero, no cuando otro programa importa sus funciones.

♥ sys.exit() y el código de salida

`sys.exit(valor)` termina el programa y devuelve un **código de salida** al sistema operativo. Por convenio, **0 = todo bien** y **distinto de 0 = error**.

```python
import sys

def main() -> int:
    print("Ejecutando el programa principal.")
    exito = True                # cambia a False para simular un error
    if exito:
        print("El programa se ejecutó correctamente.")
        return 0
    else:
        print("Ocurrió un error durante la ejecución.")
        return 1

if __name__ == "__main__":
    estado = main()
    sys.exit(estado)
```

En Linux/Mac (y en tu WSL) puedes ver el código justo después con `echo $?` (en PowerShell, `$LASTEXITCODE`). Sirve para que otros programas sepan si el tuyo terminó bien.

5.12 Hoja de ejercicios de funciones (resuelta)

EJERCICIO 1 · CALCULAR_CUADRADO

```python
def calcular_cuadrado(x):
    return x * x

n = float(input("Ingrese un número: "))
print(f"El cuadrado de {n:g} es: {calcular_cuadrado(n):g}")    El cuadrado de 5 es: 25
```

`:g` quita el ".0" sobrante al imprimir un real.

EJERCICIO 2 · AREA_CIRCULO

```python
import math
def area_circulo(radio):
    return math.pi * radio ** 2

r = float(input("Ingrese el radio del círculo: "))
print(f"El área del círculo es: {area_circulo(r):.2f}")        3 → 28.27
```

`:.2f` redondea a 2 decimales al imprimir.

EJERCICIO 3 · ES_PRIMO

Está resuelto y explicado paso a paso en el apartado 7.3.

EJERCICIO 4 · CELSIUS_A_FAHRENHEIT

```
def celsius_a_fahrenheit(c):
    return 9 / 5 * c + 32          # F = (9/5)·C + 32

c = float(input("Ingrese la temperatura en grados Celsius: "))
print("La temperatura en grados Fahrenheit es:", celsius_a_fahrenheit(c))   25 → 77.0
```

EJERCICIO 5 · CALCULAR_MCD (ALGORITMO DE EUCLIDES)

```
def calcular_mcd(a, b):
    while b != 0:
        a, b = b, a % b       # mcd(a, b) = mcd(b, a % b)
    return a
```

Traza con 56 y 98: (56, 98) → (98, 56) → (56, 42) → (42, 14) → (14, 0) → **14**. La idea de Euclides: el MCD de dos números es el mismo que el del menor y el resto de dividirlos; cuando el resto es 0, el otro número es el MCD.

EJERCICIO 6 · TIEMPO DE VIAJE Y COSTE DE COMBUSTIBLE

```
def calcular_tiempo(distancia, velocidad):
    return distancia / velocidad              # horas

def calcular_consumo(distancia, rendimiento):
    return distancia / rendimiento            # litros

def calcular_costo(consumo, precio_litro):
    return consumo * precio_litro             # euros

def main():
    distancia = float(input("Ingrese la distancia en kilómetros: "))
    velocidad = float(input("Ingrese la velocidad media en km/h: "))
    rendimiento = float(input("Ingrese el rendimiento del vehículo en km por litro: "))
    precio = float(input("Ingrese el precio del combustible por litro: "))
    tiempo = calcular_tiempo(distancia, velocidad)
    costo = calcular_costo(calcular_consumo(distancia, rendimiento), precio)
    print(f"El tiempo estimado de viaje es: {tiempo} horas")
    print(f"El costo total de combustible es: {costo:.2f}")

if __name__ == "__main__":
    main()
200, 80, 15, 1.50 → 2.5 horas · 20.00
```

6 Estructuras de datos

Este capítulo junta el tema 04 de clase y el capítulo 5 del tutorial oficial de Python (*Data Structures*), con todos sus ejemplos traducidos y explicados.

6.1 Tipos atómicos y tipos estructurados

ATÓMICOS

No se pueden descomponer en partes más pequeñas. Son **siempre inmutables** y vienen dados por los literales: `int`, `float`, `complex`, `bool`.

ESTRUCTURADOS (COMPUESTOS)

Agrupan varios valores: cadenas `str`, tuplas `tuple`, listas `list`, conjuntos `set` y diccionarios `dict`. (Se ampliará con la programación orientada a objetos.)

	¿Ordenado (secuencia)?	¿Mutable?	¿Duplicados?	Se escribe
str	Sí	No	Sí	`"hola"`
tuple	Sí	No	Sí	`(1, 'a', 3.0)`
range	Sí	No	—	`range(5)`
list	Sí	**Sí**	Sí	`[1, 'a', 3.0]`
set	No	**Sí**	**No**	`{1, 2, 3}`
dict	No (pero recuerda el orden de inserción)	**Sí**	Claves no	`{'pepe': '17/01'}`

Otros: los `bytes` son inmutables y los `bytearray`, mutables.

ANALOGÍA

Tupla = el DNI: datos fijos que no cambian (nombre, fecha de nacimiento). **Lista** = la lista de la compra: añades, tachas y cambias cosas. **Conjunto** = la colección de cromos sin repetidos. **Diccionario** = la agenda del móvil: buscas por nombre (clave) y obtienes el teléfono (valor).

¡OJO! MUTABLES DENTRO DE INMUTABLES

Una tupla es inmutable, pero si contiene una lista, **esa lista sí puede cambiar por dentro**: `t = ([1], 2)`; `t[0].append(5)` → `([1, 5], 2)`. Lo que no puedes es cambiar *qué* objetos contiene la tupla (`t[0] = [7]` da error).

6.2 Operaciones comunes a todas las secuencias

`str`, `tuple`, `list` y `range` son **secuencias**: se pueden recorrer y admiten estas operaciones (s y t son secuencias; n, i, j, k enteros; x cualquier objeto):

Operación	Resultado	Ejemplo
`x in s`	True si x está en s	`'th' in 'python'` → True
`x not in s`	lo contrario	
`s + t`	concatenación	`[1, 2] + [3]` → `[1, 2, 3]`
`s * n`, `n * s`	s repetida n veces	`'ab' * 2` → `'abab'`
`s[i]`	elemento i (empieza en 0). Si i < 0 cuenta desde el final	`'hola'[-1]` → `'a'`
`s[i:j]`	trozo (*slice*) de i a j, **sin incluir j**	`'hola'[1:3]` → `'ol'`
`s[i:j:k]`	trozo de i a j con paso k	`'python'[::2]` → `'pto'`
`len(s)`, `min(s)`, `max(s)`	longitud, menor, mayor	`min([3, 1, 2])` → 1
`s.index(x[, i[, j]])`	posición de la primera aparición de x (desde i, hasta j sin incluir)	`[1, 2, 1].index(1, 1)` → 2
`s.count(x)`	cuántas veces aparece x	`'banana'.count('a')` → 3

♥ Índices y slicing, con calma

carácter	P	Y	T	H	O	N
índice	0	1	2	3	4	5
índice negativo	-6	-5	-4	-3	-2	-1

```
s = "PYTHON"
s[0]      'P'        s[-1]     'N'       # el último, sin calcular len(s)-1
s[1:4]    'YTH'      # posiciones 1, 2 y 3 (la 4 NO)
s[:2]     'PY'       # sin inicio = desde el principio
s[2:]     'THON'     # sin fin = hasta el final
s[-3:]    'HON'      # los 3 últimos
s[::2]    'PTO'      # de 2 en 2
s[::-1]   'NOHTYP'   # ¡al revés!
s[4:1]    ''         # si inicio >= fin (con paso positivo): vacío
s[1:100]  'YTHON'    # los slices no dan error si te pasas…
s[10]     IndexError: string index out of range   # …pero los índices sí
```

ANALOGÍA · LOS CORTES ENTRE LETRAS

Imagina que los índices del slice señalan los **huecos entre letras**, no las letras: `|P|Y|T|H|O|N|` con huecos 0,1,2,3,4,5,6. `s[1:4]` corta en el hueco 1 y en el 4, y te quedas con lo de en medio: YTH. Así se ve claro por qué el fin no se incluye y por qué `s[:2] + s[2:]` siempre da la cadena entera.

6.3 Operaciones de las secuencias mutables

Solo para secuencias **mutables** (las listas): s mutable, t iterable, x objeto.

Operación	Qué hace	Equivale a
`s[i] = x`	sustituye el elemento i por x	
`s[i:j] = t`	sustituye el trozo i:j por los elementos de t (pueden ser más o menos)	
`del s[i:j]`	borra el trozo	`s[i:j] = []`
`s[i:j:k] = t`	sustituye con paso (t debe tener el mismo nº de elementos)	
`del s[i:j:k]`	borra con paso	
`s.append(x)`	añade x al final	`s[len(s):len(s)] = [x]`
`s.clear()`	vacía la lista	`del s[:]`
`s.copy()`	copia superficial	`s[:]`
`s.extend(t)`	añade al final todos los elementos de t	`s += t`
`s *= n`	la repite n veces	
`s.insert(i, x)`	inserta x en la posición i	`s[i:i] = [x]`
`s.pop([i])`	devuelve el elemento i y lo quita (sin i, el último)	
`s.remove(x)`	quita la primera aparición de x	
`s.reverse()`	la invierte en el sitio	

```
l = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
l[2:5] = ['a', 'b']      # 3 elementos sustituidos por 2
[0, 1, 'a', 'b', 5, 6, 7, 8, 9]
l = list(range(10)); del l[::2]         # borra los de posición par
[1, 3, 5, 7, 9]
l = [1, 2, 3]; l.insert(1, 'z')         [1, 'z', 2, 3]
l.pop()                                  3      → l = [1, 'z', 2]
l.pop(0)                                 1      → l = ['z', 2]
l = [1, 2, 3, 2]; l.remove(2)            [1, 3, 2]   ← solo la primera
```

6.4 Secuencias inmutables: tuplas y range

♥ Tuplas

Una tupla es un "vector" de **tamaño fijo e inmutable** que guarda datos, a menudo de tipos distintos. Lo que crea la tupla es la **coma**; los paréntesis son opcionales (salvo para la vacía):

```
x = 3, 4, 5           # tupla creada con comas
print(x)              (3, 4, 5)
x[0] == 3 and x[1] == 4 and x[2] == 5     True
x[0] = 10             TypeError: 'tuple' object does not support item assignment
(1, 'a', 3.0)         # con paréntesis
tuple("abc")          ('a', 'b', 'c')    ← con el constructor
vacia = ()            # tupla vacía
uno = 3,              (3,)   ← ¡la coma es obligatoria!
type((3))             <class 'int'>   ← sin coma, (3) es solo el número 3
```

Del tutorial oficial: las tuplas se pueden anidar y pueden contener objetos mutables:

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> u = t, (1, 2, 3, 4, 5)           # tupla de tuplas
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> v = ([1, 2, 3], [3, 2, 1])       # tupla que contiene listas
>>> empty = ()
>>> singleton = 'hello',             # <-- fíjate en la coma final
>>> len(empty), len(singleton)
(0, 1)
>>> x, y, z = t                      # desempaquetado de secuencias
```

`t = 12345, 54321, 'hello!'` es un **empaquetado** de tupla; `x, y, z = t` es lo contrario, un **desempaquetado**: necesita tantas variables como elementos. La asignación múltiple (`a, b = 1, 2`) es justo esto: empaquetar y desempaquetar.

¿TUPLA O LISTA?

Las tuplas suelen guardar datos **heterogéneos** (un registro: nombre, edad, nota) y se usan con desempaquetado o índices. Las listas suelen guardar datos **homogéneos** (muchas notas) y se recorren con bucles.

♥ range

Representa una secuencia inmutable de números; se usa sobre todo en los `for`. `range(fin)`, `range(inicio, fin)`, `range(inicio, fin, paso)`; inicio vale 0 y paso 1 si no se indican.

- Paso **positivo**: contiene `r[i] = inicio + i·paso` mientras `r[i] < fin`.
- Paso **negativo**: contiene `r[i] = inicio + i·paso` mientras `r[i] > fin`.

```
list(range(0, -10, -3))     [0, -3, -6, -9]
range(5)[2]                 2       ← también se puede indexar
list(range(5, 1))           []      ← con paso positivo y inicio > fin, vacío
```

6.5 Cadenas: métodos

En el intérprete, escribe `"".` y pulsa Tab para ver todos los métodos. Los más útiles (recuerda que las cadenas son inmutables: **los métodos devuelven una cadena nueva**, no cambian la original):

Método	Qué hace	Ejemplo
`s.find(sub[, ini[, fin]])`	índice más bajo donde aparece sub (en s[ini:fin]); -1 si no está	`"hola mundo".find("mundo")` → 5
`s.join(iterable)`	une las cadenas del iterable usando s como separador	`"-".join(["2026","10","22"])` → `'2026-10-22'`
`s.replace(vieja, nueva[, n])`	sustituye todas (o solo n) apariciones	`"banana".replace("a","o",2)` → `'bonona'`
`s.split(sep=None, maxsplit=-1)`	lista de trozos separando por sep (sin sep: por espacios); como mucho maxsplit cortes	`"a,b,,c".split(",")` → `['a','b','','c']`
`s.format(…)`	crea cadenas con formato	`"{} tiene {}".format("Ana", 20)`
`s.upper()` / `s.lower()`	a mayúsculas / minúsculas	`"Hola".upper()` → `'HOLA'`
`s.strip()`	quita espacios al principio y al final	`" x ".strip()` → `'x'`
`s.startswith(p)` / `s.endswith(p)`	¿empieza / acaba por p?	`"hola".startswith("ho")` → True
`s.isdigit()` / `s.isalpha()`	¿son todo dígitos / todo letras?	`"123".isdigit()` → True

♥ Formatear cadenas

```
"Nombre: {}, edad: {}".format("Ana", 20)   'Nombre: Ana, edad: 20'
"{1} {0}".format("a", "b")                  'b a'          ← por posición
"{:.3f}".format(3.14159)                    '3.142'        ← 3 decimales
nombre = "Ana"
f"Hola {nombre}, 2+3={2+3}"                 'Hola Ana, 2+3=5'   ← f-string (la más cómoda)
```

6.6 Listas a fondo

Secuencias **mutables** que pueden guardar elementos de cualquier tipo. Formas de crearlas:

- `[]` → lista vacía · `[1, 'a', 3.0, -5+7j]` → con elementos.
- `list()` o `list(iterable)` → `list("abc")` da `['a', 'b', 'c']`.
- Por **comprensión**: `[x for x in iterable]` (apartado 6.8).
- Son muy habituales las **listas anidadas**, por ejemplo para una matriz: `m = [[1, 2, 3], [4, 5, 6]]`; `m[1][2]` → 6 (fila 1, columna 2).

♥ Todos los métodos de las listas (tutorial oficial)

Método	Qué hace
`list.append(x)`	Añade x al final. Como `a[len(a):] = [x]`.
`list.extend(iterable)`	Añade todos los elementos del iterable. Como `a[len(a):] = iterable`.
`list.insert(i, x)`	Inserta x **antes** de la posición i. `a.insert(0, x)` lo pone al principio; `a.insert(len(a), x)` equivale a append.
`list.remove(x)`	Quita el primer elemento igual a x. `ValueError` si no existe.
`list.pop([i])`	Quita y **devuelve** el elemento i (sin i, el último). `IndexError` si está vacía o i no existe.
`list.clear()`	Lo quita todo. Como `del a[:]`.
`list.index(x[, ini[, fin]])`	Posición de la primera x (buscando entre ini y fin; la posición devuelta es respecto a la lista entera). `ValueError` si no está.
`list.count(x)`	Cuántas veces aparece x.
`list.sort(*, key=None, reverse=False)`	Ordena la lista **en el sitio**.
`list.reverse()`	Invierte la lista en el sitio.
`list.copy()`	Copia superficial. Como `a[:]`.

EJEMPLO DEL TUTORIAL

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.count('tangerine')
0
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)      # busca el siguiente banana a partir de la posición 4
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'
```

¡OJO! LOS MÉTODOS QUE MODIFICAN DEVUELVEN NONE

`insert`, `remove`, `sort`, `reverse`… cambian la lista y **devuelven None** (es un principio de diseño de Python para los tipos mutables). Así que `l = l.sort()` te deja `l = None`. Usa `l.sort()` a secas, o `nueva = sorted(l)` si quieres una copia ordenada.

Además, no todo se puede ordenar: `[None, 'hello', 10]` da TypeError (no se pueden comparar tipos distintos), y los complejos tampoco tienen orden (`3+4j < 5+7j` no es válido).

♥ Recorrer una lista

```
lista = [10, 20, 30]
# 1) Directamente
for v in lista:
    print(v)
```

```
# 2) Con un índice
for i in range(len(lista)):
    print(lista[i])
```

```
# 3) Índice y valor
for i, v in enumerate(lista):
    print(i, v)
0 10 · 1 20 · 2 30
```

♥ Usar una lista como pila

Una **pila** es LIFO (*last in, first out*): el último que entra es el primero que sale. Con una lista: `append()` para meter (push) y `pop()` sin índice para sacar.

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```

♥ Usar una lista como cola: mejor deque

Una **cola** es FIFO (*first in, first out*): como la fila del súper. Con una lista es **lento**, porque quitar o meter por el principio obliga a mover todos los demás elementos. Para eso existe `collections.deque`, rápida por los dos extremos:

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # llega Terry
>>> queue.append("Graham")          # llega Graham
>>> queue.popleft()                 # sale el primero que llegó
'Eric'
>>> queue.popleft()
'John'
>>> queue
deque(['Michael', 'Terry', 'Graham'])
```

ANALOGÍA

Pila = una torre de platos (coges el de arriba, el último que pusiste). **Cola** = la cola de la cafetería (atienden primero al que llegó antes).

6.7 La sentencia del

Borra elementos por **índice**, por **trozo** o borra una **variable** entera. A diferencia de `pop()`, no devuelve nada.

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
>>> del a           # borra la variable: usar a después da NameError
```

6.8 Comprensiones de listas

Una **comprensión** crea una lista en una sola línea a partir de un bucle (y opcionalmente un filtro). Estas dos formas hacen lo mismo:

```
cuadrados = []
for x in range(10):
    cuadrados.append(x**2)
```

```
cuadrados = [x**2 for x in range(10)]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
# también: list(map(lambda x: x**2, range(10)))
```

La versión con `for` además deja creada la variable `x` (con el valor 9) cuando termina; la comprensión no.

ESTRUCTURA

`[ expresión for variable in iterable if condición ]`

"Dame *expresión* para cada *variable* de *iterable* que cumpla *condición*". Puede haber varios `for` e `if`.

ANALOGÍA

Es como pedirle a alguien: "de esta cesta de fruta (iterable), coge las que estén maduras (if) y córtalas en trozos (expresión)". Todo en una frase.

Ejemplos del tutorial:

```
>>> vec = [-4, -2, 0, 2, 4]
>>> [x*2 for x in vec]                # nueva lista con los valores duplicados
[-8, -4, 0, 4, 8]
>>> [x for x in vec if x >= 0]         # filtrar para quitar los negativos
[0, 2, 4]
>>> [abs(x) for x in vec]              # aplicar una función a todos
[4, 2, 0, 2, 4]
>>> freshfruit = ['  banana', '  loganberry ', 'passion fruit  ']
>>> [weapon.strip() for weapon in freshfruit]   # llamar a un método en cada uno
['banana', 'loganberry', 'passion fruit']
>>> [(x, x**2) for x in range(6)]        # crear tuplas (x, x²)
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
>>> [x, x**2 for x in range(6)]          # sin paréntesis en la tupla: error
SyntaxError
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]   # aplanar una lista de listas
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> from math import pi
>>> [str(round(pi, i)) for i in range(1, 6)]
['3.1', '3.14', '3.142', '3.1416', '3.14159']
```

EJEMPLO · DOS FOR Y UN IF

```
>>> [(x, y) for x in [1,2,3]
...         for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1),
 (2, 4), (3, 1), (3, 4)]
```

```
combs = []
for x in [1,2,3]:
    for y in [3,1,4]:
        if x != y:
            combs.append((x, y))
```

Los `for` y el `if` van **en el mismo orden** que en la versión con bucles.

♥ Comprensiones anidadas: trasponer una matriz

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
>>> [[row[i] for row in matrix] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

La comprensión de dentro se evalúa para cada `i`: "coge el elemento i de cada fila" → esa es la columna i, que pasa a ser la fila i. Con bucles normales:

```
transposed = []
for i in range(4):
    transposed_row = []
    for row in matrix:
        transposed_row.append(row[i])
    transposed.append(transposed_row)
```

En la práctica es mejor usar las funciones ya hechas: `list(zip(*matrix))` → `[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]`.

♥ Desempaquetar con * dentro de listas

```
>>> x = [1, 2, 3]
>>> [0, *x, 4, 5, 6]        # *x "vuelca" los elementos de x
[0, 1, 2, 3, 4, 5, 6]
>>> x = 1
>>> [0, *x, 2]
TypeError: Value after * must be an iterable, not int
```

¡OJO! NOVEDAD MUY RECIENTE

La versión actual del tutorial también muestra `[*element for element in x]` para aplanar listas. Es una novedad de las versiones **más recientes** de Python, posteriores a la 3.13: en Python 3.13 da `SyntaxError: iterable unpacking cannot be used in comprehension`. Para aplanar en cualquier versión usa `[num for elem in x for num in elem]`.

6.9 Conjuntos (set)

Un conjunto es una colección **mutable**, **no ordenada** y **sin duplicados**. Sirve para comprobar si algo pertenece y para quitar repetidos, y admite las operaciones de conjuntos de mates.

- Conjunto vacío: `set()`. **¡No `{}`!**, que crea un diccionario vacío.
- Con llaves: `{1, 2, 3, 1}` → `{1, 2, 3}` (el repetido desaparece).
- Por comprensión: `{x for x in 'abracadabra' if x not in 'abc'}` → `{'r', 'd'}`.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> print(basket)                      # los duplicados se han eliminado
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                 # comprobar pertenencia: muy rápido
True
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                  # letras distintas de a
{'a', 'r', 'b', 'c', 'd'}
```

Operación	Nombre	Resultado con a y b
`a - b`	diferencia	letras de a que no están en b: `{'r', 'd', 'b'}`
`a | b`	unión	letras de a, de b o de las dos: `{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}`
`a & b`	intersección	letras de a y de b a la vez: `{'a', 'c'}`
`a ^ b`	diferencia simétrica	letras de a o de b pero no de las dos: `{'r', 'd', 'b', 'm', 'z', 'l'}`

Otros métodos útiles: `s.add(x)` añade, `s.remove(x)` quita (error si no está), `s.discard(x)` quita sin error, `a <= b` pregunta si a es subconjunto de b.

¡OJO!

El orden en que se imprimen los elementos de un conjunto **no está garantizado** (puede salirte distinto que aquí). Y los conjuntos no admiten índices: `basket[0]` da error.

6.10 Diccionarios (dict)

Un diccionario es una colección de pares **clave: valor**. Se accede a los valores **por su clave**, no por posición.

- Las **claves** deben ser **inmutables**: cadenas, números o tuplas que solo contengan cosas inmutables. Una lista no puede ser clave. Los **valores** pueden ser de cualquier tipo.
- Las claves son **únicas**: si guardas con una clave que ya existe, se pierde el valor anterior.

♥ Formas de crearlo

```
cumples = {'pepe': '17/01', 'juan': '23/02'}            # con llaves
vacio = {}                                              # o dict()
dict([('pepe', '17/01'), ('juan', '23/02')])            # desde una lista de pares
dict(sape=4139, guido=4127, jack=4098)                  # con argumentos por nombre (claves = ca
{x: x**2 for x in (2, 4, 6)}                          {2: 4, 4: 16, 6: 36}   ← por comprensión
print(cumples['pepe'])                                  17/01
```

♥ Operaciones (tutorial oficial)

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127              # añadir (o modificar)
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']                      # consultar
4098
>>> tel['irv']                       # clave que no existe
KeyError: 'irv'
>>> print(tel.get('irv'))            # get() no da error: devuelve None (o un valor por defecto)
None
>>> del tel['sape']                  # borrar un par
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)                        # las claves, en orden de inserción
['jack', 'guido', 'irv']
>>> sorted(tel)                      # las claves ordenadas
['guido', 'irv', 'jack']
>>> 'guido' in tel                   # ¿existe la clave?
True
>>> 'jack' not in tel
False
>>> odds = {i: i**2 for i in (1, 3, 5)}
>>> evens = {i: i**2 for i in (2, 4, 6)}
>>> {**odds, **evens}                # juntar dos diccionarios
{1: 1, 3: 9, 5: 25, 2: 4, 4: 16, 6: 36}
```

Más métodos: `d.keys()`, `d.values()`, `d.items()` (pares), `d.pop(clave)` (quita y devuelve), `d.get(clave, defecto)`, `len(d)`.

EJEMPLO TÍPICO · CONTAR APARICIONES

```
texto = "hola mundo"
cuenta = {}
for letra in texto:
    cuenta[letra] = cuenta.get(letra, 0) + 1     # si no estaba, empieza en 0
print(cuenta['o'])        2
```

6.11 Técnicas de bucles

Quiero recorrer…	Uso
las **claves** de un diccionario	`for k in d:`
los **valores**	`for k in d: print(d[k])` · o `for v in d.values():`
clave y valor a la vez	`for k, v in d.items():`
posición y valor de una secuencia	`for i, v in enumerate(s):`
dos o más secuencias en paralelo	`for a, b in zip(s1, s2):`
una secuencia **al revés**	`for x in reversed(s):`
una secuencia **ordenada** (sin cambiar la original)	`for x in sorted(s):`
los elementos **sin repetir**, en orden	`for x in sorted(set(s)):`

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():
...     print(k, v)
gallahad the pure
robin the brave
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
0 tic
1 tac
2 toe
>>> questions = ['name', 'quest', 'favorite color']
>>> answers = ['lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print('What is your {0}?  It is {1}.'.format(q, a))
What is your name?  It is lancelot.
What is your quest?  It is the holy grail.
What is your favorite color?  It is blue.
>>> for i in reversed(range(1, 10, 2)):
...     print(i)
9 7 5 3 1 (uno por línea)
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> for f in sorted(set(basket)):
...     print(f)
apple banana orange pear (uno por línea)
```

¡OJO! NO MODIFIQUES UNA LISTA MIENTRAS LA RECORRES

Es más fácil y seguro **crear una lista nueva**. Ejemplo del tutorial: quitar los valores NaN ("no es un número") de unos datos:

```
import math
raw_data = [56.2, float('NaN'), 51.7, 55.3, 52.5, float('NaN'), 47.8]
filtered_data = []
for value in raw_data:
    if not math.isnan(value):
        filtered_data.append(value)
filtered_data         [56.2, 51.7, 55.3, 52.5, 47.8]
```

(O en una línea: `[v for v in raw_data if not math.isnan(v)]`.)

6.12 Más sobre condiciones

- `in` / `not in` comprueban si un valor está (o no) en un contenedor. `is` / `is not` comprueban si dos objetos son **el mismo**.
- Todas las comparaciones tienen la misma prioridad, menor que la de los operadores numéricos.
- Se pueden encadenar: `a < b == c` comprueba si a es menor que b **y además** b es igual a c.
- `and`, `or` y `not` tienen menos prioridad que las comparaciones; entre ellos, `not` es el más fuerte y `or` el más débil: `A and not B or C` = `(A and (not B)) or C`.
- `and` y `or` son de **cortocircuito**: evalúan de izquierda a derecha y paran en cuanto saben el resultado. Además devuelven **el último valor evaluado**, no necesariamente True/False:

```
>>> string1, string2, string3 = '', 'Trondheim', 'Hammer Dance'
>>> non_null = string1 or string2 or string3     # el primero que no esté vacío
>>> non_null
'Trondheim'
```

En Python, a diferencia de C, no se puede asignar dentro de una expresión con `=` (evita el error de escribir `=` queriendo `==`). Si de verdad lo necesitas, existe el operador **morsa** `:=`: `while (linea := input()) != "":`.

6.13 Comparar secuencias

Las secuencias del mismo tipo se comparan en **orden lexicográfico** (como en el diccionario): se comparan los primeros elementos; si son iguales, los segundos; y así. Si una es el **principio** de la otra, la más corta es la menor. Las cadenas se comparan por el código Unicode de cada carácter (por eso las mayúsculas van antes que las minúsculas).

```
(1, 2, 3)              < (1, 2, 4)                 True
[1, 2, 3]              < [1, 2, 4]                 True
'ABC' < 'C' < 'Pascal' < 'Python'                  True
(1, 2, 3, 4)           < (1, 2, 4)                 True    ← decide el 3 < 4
(1, 2)                 < (1, 2, -1)                True    ← la más corta es prefijo
(1, 2, 3)             == (1.0, 2.0, 3.0)           True    ← 1 == 1.0
(1, 2, ('aa', 'ab'))   < (1, 2, ('abc', 'a'), 4)   True    ← se compara dentro: 'aa' < 'abc'
1 < "a"                                            TypeError ← tipos que no se pueden comparar
```

Los números de tipos distintos se comparan por su valor (`0 == 0.0`). Comparar otros tipos distintos con `<` da `TypeError`.

RESUMEN DEL CAPÍTULO 6

Secuencias (str, tuple, range, list): índices desde 0, negativos desde el final, slicing sin incluir el fin · tuplas y cadenas inmutables, listas mutables · los métodos que modifican devuelven None · pila = append/pop, cola = deque · comprensión = `[expr for x in it if cond]` · set: sin repetidos, sin orden, `set()` para el vacío · dict: clave inmutable → valor, `get()` para no fallar · recorre con `items()`, `enumerate()`, `zip()`, `reversed()`, `sorted()`.

7 Programas con estructuras de datos

Los cinco ejemplos que aparecen en el aula virtual (semana 6), resueltos y explicados paso a paso. Todos están probados.

7.1 Suma de los n primeros cuadrados (recursiva)

ENUNCIADO

Leer un número entero positivo n y devolver la suma de los n primeros cuadrados: $1^2 + 2^2 + \ldots + n^2$. Hacerlo recursivamente.

Cómo pensarlo (receta del apartado 5.10): **caso base**: si n = 0, la suma es 0 (no hay nada que sumar). **Caso recursivo**: la suma hasta n es n^2 más la suma hasta n-1.

```
def suma_cuadrados(n):
    """Devuelve 1² + 2² + ... + n² de forma recursiva."""
    if n == 0:                              # caso base
        return 0
    return n ** 2 + suma_cuadrados(n - 1)    # caso recursivo

def main():
    n = int(input("Introduce un entero positivo: "))
    print("La suma de los", n, "primeros cuadrados es", suma_cuadrados(n))

if __name__ == "__main__":
    main()
```

Traza para n = 3: `suma_cuadrados(3)` = 9 + `suma_cuadrados(2)` = 9 + (4 + `suma_cuadrados(1)`) = 9 + 4 + (1 + `suma_cuadrados(0)`) = 9 + 4 + 1 + 0 = **14**. Con n = 10 da 385.

PARA COMPROBARLO

Existe la fórmula n(n+1)(2n+1)/6. Para n = 3: 3·4·7/6 = 14 ✓. Comparar tu función con la fórmula es una buena forma de **probar** el programa.

7.2 Divisores propios de un número natural

ENUNCIADO

Programar una función que calcule los divisores propios de un número natural (todos sus divisores salvo él mismo).

Idea: d es divisor de n si el resto de n entre d es 0 (`n % d == 0`). Probamos todos los d desde 1 hasta n-1 y guardamos los que cumplen.

```python
def divisores_propios(n):
    """Lista de divisores de n
    menores que n."""
    divisores = []
    for d in range(1, n):
        if n % d == 0:
            divisores.append(d)
    return divisores
```

```python
# La misma idea con una comprensión
def divisores_propios(n):
    return [d for d in range(1, n)
            if n % d == 0]

divisores_propios(12)  [1, 2, 3, 4, 6]
divisores_propios(7)   [1]
divisores_propios(1)   []
```

Mejora opcional: ningún divisor propio (salvo n) es mayor que n/2, así que basta `range(1, n // 2 + 1)` (para n ≥ 2). Hace la mitad de trabajo.

7.3 ¿Es primo un número natural?

ENUNCIADO

Escribir una función que diga si un número natural es primo (también es el ejercicio 3 de la hoja de funciones).

Un número es primo si es **mayor que 1** y sus únicos divisores son 1 y él mismo.

```python
def es_primo(n):
    """True si n es primo, False si no."""
    if n < 2:                  # 0 y 1 no son primos
        return False
    d = 2
    while d * d <= n:          # basta probar hasta la raíz cuadrada de n
        if n % d == 0:
            return False       # encontrado un divisor: no es primo
        d += 1
    return True                # ningún divisor: es primo

n = int(input("Ingrese un número: "))
print(f"El número {n} es primo: {es_primo(n)}")     7 → El número 7 es primo: True
```

¿POR QUÉ BASTA LLEGAR A LA RAÍZ?

Si n = a·b y los dos fueran mayores que √n, su producto sería mayor que n. Así que si n tiene un divisor, alguno es ≤ √n. Para 97 basta probar del 2 al 9 en vez del 2 al 96. Versión sencilla (más lenta pero correcta): `return n > 1 and divisores_propios(n) == [1]`.

Primos menores que 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 (comprobado con `[p for p in range(30) if es_primo(p)]`).

7.4 Parejas de números amigos

ENUNCIADO

Tomar un número natural por la entrada estándar y devolver todas las parejas de números amigos (hasta ese número). Dos números son amigos si cada uno es igual a la suma de los divisores propios del otro.

Ejemplo clásico: 220 y 284. Divisores propios de 220: 1+2+4+5+10+11+20+22+44+55+110 = **284**. Divisores propios de 284: 1+2+4+71+142 = **220**. ¡Amigos!

Idea: para cada a, calculo b = suma de divisores de a. Si la suma de divisores de b vuelve a ser a, son amigos. Pido **a < b** para no repetir la pareja (220, 284) y (284, 220), y para descartar los números **perfectos** (como 6 = 1+2+3, que sería "amigo de sí mismo").

```python
def divisores_propios(n):
    return [d for d in range(1, n) if n % d == 0]

def suma_divisores(n):
    return sum(divisores_propios(n))

def parejas_amigos(limite):
    """Lista de tuplas (a, b) de números amigos con a < b <= limite."""
    parejas = []
    for a in range(2, limite + 1):
        b = suma_divisores(a)
        if a < b <= limite and suma_divisores(b) == a:
            parejas.append((a, b))
    return parejas

def main():
    limite = int(input("Introduce un número natural: "))
    for a, b in parejas_amigos(limite):
        print(a, "y", b, "son amigos")

if __name__ == "__main__":
    main()
```

Con 1500 imprime *220 y 284* y *1184 y 1210*. Con 10000 salen además (2620, 2924), (5020, 5564) y (6232, 6368).

Fíjate en cómo se reutilizan funciones: `parejas_amigos` usa `suma_divisores`, que usa `divisores_propios`. Cada una hace una sola cosa (alta cohesión).

7.5 De decimal a números romanos (menor que 4000)

ENUNCIADO

Escribir un programa que transforme un número decimal menor que 4000 a su expresión en números romanos.

Idea: una lista de pares (valor, letras) ordenada de mayor a menor, que **incluye los casos de resta** (900 = CM, 400 = CD, 90 = XC, 40 = XL, 9 = IX, 4 = IV). Para cada par, mientras el número sea mayor o igual que el valor, se añaden las letras y se resta el valor. Es como pagar con el menor número de billetes: siempre usas el más grande que cabe.

```python
def a_romano(n):
    """Convierte un entero 1 <= n < 4000 a número romano."""
    valores = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"),
               (100, "C"), (90, "XC"), (50, "L"), (40, "XL"),
               (10, "X"), (9, "IX"), (5, "V"), (4, "IV"), (1, "I")]
    resultado = ""
    for valor, letras in valores:      # desempaquetado de cada tupla
        while n >= valor:
            resultado += letras
            n -= valor
    return resultado

def main():
    n = int(input("Número decimal (1-3999): "))
    if 1 <= n < 4000:
        print(a_romano(n))
    else:
        print("El número debe estar entre 1 y 3999")

if __name__ == "__main__":
    main()
```

Paso (n = 1994)	Se añade	Queda
1994 ≥ 1000	M	994
994 ≥ 900	CM	94
94 ≥ 90	XC	4
4 ≥ 4	IV	0
MCMXCIV		

Otros: 4 → IV · 2026 → MMXXVI · 3999 → MMMCMXCIX (el máximo, porque no hay símbolo para 5000; por eso el enunciado pide menor que 4000).

EJERCICIO DE PROGRAMACIÓN 2

En la captura del aula virtual aparece como un punto aparte, pero su enunciado no estaba entre los PDFs que me pasaste. Si me lo mandas, lo resolvemos igual que estos.

8 Chuleta

Entrada y salida

```
n = int(input("Texto: "))     # leer entero
x = float(input())            # leer real
print("a", 3, sep="-", end="") # a-3
print(f"{x:.2f}")             # 2 decimales
```

Tipos y conversión

```
int  float  complex  bool  str  None
type(x)   int("5")  str(5)  float("3.5")
bin(10) hex(255) int("ff", 16)
```

Operadores

```
+ - * /  //(cociente) %(resto) **(potencia)
== != < <= > >=  is  in
and  or  not        # cortocircuito
x += 1  (y -=, *=, /=, //=, %=, **=)
```

Falsos

```
False None 0 0.0 '' () [] {} set() range(0)
```

Control

```
if c1:  ...  elif c2:  ...  else:  ...
for x in secuencia: ...
for i in range(ini, fin, paso): ...  # sin fin
while cond: ...
break   # salir      continue  # siguiente vuelta
```

Funciones

```
def f(a, b=1, *args, c, **kw) -> int:
    """docstring"""
    return a + b
f(1, 2, c=3)        # posición y nombre
global x / nonlocal x
lambda x, y: x * y
if __name__ == "__main__": main()
```

Secuencias (str, tuple, list, range)

```
s[0]  s[-1]  s[i:j]  s[::-1]  len(s)
x in s  s + t  s * n  min max  s.index(x)
s.count(x)
```

Cadenas

```
s.split(",")  ",".join(lista)  s.strip()
s.upper() s.lower() s.replace(a, b) s.find(x)
```

Listas

```
l.append(x)  l.extend(t)  l.insert(i, x)
l.pop()  l.pop(i)  l.remove(x)  del l[i]
l.sort()  sorted(l)  l.reverse()  l.copy()
[x**2 for x in l if x > 0]     # comprensión
```

Tuplas

```
t = (1, 'a')   uno = (3,)   a, b = t
```

Conjuntos

```
s = set()   s = {1, 2}   s.add(x)  s.discard(x)
a | b   a & b   a - b   a ^ b
```

Diccionarios

```
d = {'k': 1}   d['k2'] = 2   del d['k']
d.get(k, defecto)   k in d   d.keys() d.values()
for k, v in d.items(): ...
{x: x**2 for x in range(5)}
```

Bucles útiles

```
for i, v in enumerate(l): ...
for a, b in zip(l1, l2): ...
for x in reversed(l): ...
for x in sorted(set(l)): ...
```

Patrones

```
suma = 0 → suma += x        # acumulador
prod = 1 → prod *= x
cont = 0 → if cond: cont += 1  # contador
n % 2 == 0   # par     n % 10  # última cifra
```

9 Erratas corregidas

Esto es lo que estaba mal o era confuso en el material, y cómo aparece en la guía (ya corregido en su sitio). Si en clase os lo explicaron de otra forma, haz caso al profe. ¡Mucho ánimo! ♥

Dónde	Pone	Correcto
03 Funciones · parámetros por defecto	Se define `son_iguales` pero se llama `sonIguales(...)`	El nombre debe coincidir: `son_iguales(0.999, 1.0)` (si no, `NameError`)
03 Funciones · nonlocal	Dentro de `sy()` pone "local de gy" y "Valor de y en gy"	La función se llama `sy`: "local de sy"
04 Estructuras · tuplas	"`3,` genera la tupla (3)"	Genera `(3,)`. `(3)` sin coma es el entero 3
04 Estructuras · conjuntos	"`1,2,3,1` creará el conjunto `2,1,3`" (sin llaves)	`{1, 2, 3, 1}` crea `{1, 2, 3}`; faltaban las llaves y el orden de un conjunto no está definido
04 Estructuras · conjuntos y dict	Comprensiones escritas sin llaves: `x for x in 'abracadabra' if …`, `x: x**2 for x in (2,4,6)`	`{x for x in 'abracadabra' if x not in 'abc'}` y `{x: x**2 for x in (2, 4, 6)}`
04 Estructuras · diccionarios	"A los elementos de un **conjunto** se accede por su clave"	De un **diccionario**; los conjuntos no tienen claves ni índices
04 Estructuras · cadenas	"Poner en el intérprete ". y esperar"	Escribir `"".` y pulsar Tab para ver los métodos
02 Sentencias · for y while	Comentarios con `//Imprimir…`	En Python los comentarios empiezan por `#`; `//` es de C (en Python es la división entera)
02 Sentencias · if anidados	Líneas acabadas en `;`	Python no los necesita (funcionan, pero no se usan)
01 Introducción · variables	`x = x + 1;`	`x = x + 1`
01 Introducción · booleanos	Lista de vacíos: `",(),[],, set()`	`'', (), [], {}, set(), range(0)`
01 Introducción · literales	`inf` como literal float	No existe ese literal: se escribe `float('inf')` o `math.inf`
Cuaderno 03 · notas	Una nota negativa da "Suspenso"	Comprobar primero que la nota esté entre 0 y 10
Cuaderno 03 · sumas	`sum = 0`	Funciona, pero tapa la función `sum()`: mejor `suma`
Cuaderno 02 · cadenas	`print("c:\home\nombreusuario")`	Imprime mal por `\n` (y Python avisa por `\h`): usar `r"c:\home\nombreusuario"` o `\\`
Hoja de ejercicios de funciones	F = 95 C + 32	F = (9/5)·C + 32 (la fracción se perdió al copiar)
docs.python.org · 5.1.5	`[*element for element in x]`	Solo funciona en versiones de Python posteriores a la 3.13; en 3.13 da SyntaxError. Alternativa: `[n for e in x for n in e]`

Varios	"complento", "reemplaa", "Sequencias", "longintud", "asterísco", "inclinida", "nómbres", "expressión", "suando", "sustiturir", "Sentecias", "varlor_izq", "crerá", "compresión"	complemento, reemplaza, Secuencias, longitud, asterisco, inclinada, nombres, expresión, usando, sustituir, Sentencias, valor_izq, creará, **comprensión** (de listas)